HANDS-ON-HANDBUCH

Linkerd zum Anfassen

Service Mesh in der Praxis: vom ersten Sidecar bis zum Betrieb



Thomas Zachmann

Stand: 6. November 2026

Linkerd zum Anfassen

Service Mesh in der Praxis: vom ersten Sidecar bis zum Betrieb

Thomas Zachmann

Stand: 6. November 2026 · Linkerd edge-26.11.1 (Linie 2.20)

Linkerd zum Anfassen · 1. Auflage, November 2026 · © 2026 Thomas Zachmann

Kostenlos unter der Lizenz CC BY-ND 4.0: Weitergabe erlaubt, unverändert und mit Namensnennung. creativecommons.org/licenses/by-nd/4.0

Labs und Prüfprotokolle: github.com/thomaszachmann/linkerd-zum-anfassen

Linkerd ist ein Projekt der Cloud Native Computing Foundation. Dieses Buch ist unabhängig und steht in keiner Verbindung zum Linkerd-Projekt oder zu Buoyant, Inc.

Prüfstand: Die Labs liefen am 6. November 2026 mit edge-26.11.1 in einem kind-Cluster. Anhang B nennt die Testumgebung und die offenen Punkte.

Inhalt

Vorwort **6**
So arbeitest du mit diesem Buch 6
Was du über den Prüfstand wissen musst 7
Versionen 7
1 Warum ein Service Mesh? **8**
1.1 Das Problem steckt zwischen den Diensten 8
1.2 Die Idee: ein Proxy neben jedem Pod 8
1.3 Was Linkerd anders macht 9
1.4 Wann du kein Service Mesh brauchst 9
1.5 Zusammenfassung 10
2 Das Lab aufbauen **12**
2.1 Der Cluster 12
2.2 Die CLI 13
2.3 Gateway API zuerst 13
2.4 Die Control Plane 14
2.5 Die Viz-Erweiterung 15
2.6 Aufräumen 15
2.7 Zusammenfassung 15
3 Wie Linkerd aufgebaut ist **16**
3.1 Das Gesamtbild 16
3.2 Control Plane 16
3.3 Data Plane 17
3.4 Outbound und Inbound 18
3.5 Protokollerkennung 18
3.6 Zusammenfassung 18
4 Die erste Anwendung im Mesh **20**
4.1 Emojivoto 20
4.2 Ins Mesh nehmen 20
4.3 Der erste Blick 21
4.4 Den Fehler finden 21
4.5 Zusammenfassung 22
5 mTLS und Identität **24**

5.1 Identität statt IP-Adresse ... 24
5.2 Was beim Start eines Proxys passiert ... 24
5.3 Den Nachweis führen ... 24
5.4 Die wichtigste Einschränkung ... 26
5.5 Zusammenfassung ... 27
6 Autorisierung ... 28
6.1 Das Entscheidungsmodell ... 28
6.2 Die Default-Policy ... 29
6.3 Lab: den Voting-Dienst abschotten ... 30
6.4 Lab: Autorisierung pro Route ... 33
6.5 Zusammenfassung ... 36
7 Zertifikate für den Ernstfall ... 38
7.1 Die Vertrauenskette ... 38
7.2 Lab: eigene Zertifikate mit step ... 39
7.3 Automatische Rotation mit cert-manager ... 40
7.4 Zusammenfassung ... 41
8 Beobachten ... 42
8.1 Die goldenen Signale ... 42
8.2 Werkzeuge in der CLI ... 42
8.3 Rohe Metriken ... 43
8.4 Prometheus in der Produktion ... 44
8.5 Zusammenfassung ... 44
9 Retries und Timeouts ... 46
9.1 Der Aufbau ... 46
9.2 Retries ... 49
9.3 Timeouts ... 51
9.4 Fallstricke ... 52
9.5 Zusammenfassung ... 53
10 Circuit Breaking und Rate Limiting ... 54
10.1 Lab: Circuit Breaking ... 54
10.2 Lab: Rate Limiting ... 58
10.3 Rate-Limit-bewusstes Load Balancing ... 60
10.4 Zusammenfassung ... 60
11 Traffic Shifting ... 62
11.1 Das Prinzip ... 62
11.2 Lab: zehn Prozent auf die neue Version ... 65

11.3 Lab: Dark Launch per Header ... 66
11.4 Automatisieren ... 67
11.5 Aufräumen ... 67
11.6 Zusammenfassung ... 68
12 Ingress und Egress ... 70
12.1 Ingress ... 70
12.2 Egress ... 71
12.3 Zusammenfassung ... 74
13 Multicluster im Überblick ... 76
13.1 Das Modell ... 76
13.2 Der Ablauf ... 76
13.3 Mit kind ausprobieren ... 77
13.4 Federated Services ... 78
13.5 Vor dem Start ... 78
13.6 Zusammenfassung ... 79
14 Betrieb ... 80
14.1 Das Release-Modell ... 80
14.2 Installation mit Helm ... 80
14.3 Was der HA-Modus tut ... 81
14.4 Upgrades ... 82
14.5 Checkliste für die Produktion ... 83
14.6 Zusammenfassung ... 83
15 Fehlersuche ... 84
15.1 Das Vorgehen ... 84
15.2 Häufige Fehlerbilder ... 84
15.3 Der 10-Sekunden-Fall ... 85
15.4 Mehr Details aus dem Proxy ... 86
15.5 Eine Übung zum Abschluss ... 87
15.6 Zusammenfassung ... 87
Anhang A: Spickzettel ... 88
Installation ... 88
Pods ins Mesh nehmen ... 88
Diagnose ... 88
Annotationen ... 88
Anhang B: Prüfstand dieser Auflage ... 90
Testumgebung ... 90

Im Cluster gelaufen 90
In der ersten Fassung falsch 91
Nicht gelaufen oder offen 92
Abweichungen von der Linkerd-Doku 93
Selbst nachprüfen 93
Anhang C: Quellen 96
Linkerd 96
Weitere 97

Vorwort

Ein Service Mesh lernt man nicht aus Architekturdiagrammen. Man versteht es in dem Moment, in dem ein Request mit `403` abprallt, obwohl der Pod läuft und der Service stimmt, und man herausfinden muss, welche Regel gerade zugeschlagen hat. Deshalb ist dieses Buch ein Arbeitsbuch. Du baust dir ein lokales Lab, nimmst eine Beispielanwendung ins Mesh und machst sie Kapitel für Kapitel sicherer, robuster und besser beobachtbar. Dabei geht auch absichtlich etwas kaputt.

Das Buch richtet sich an Menschen, die Kubernetes im Alltag benutzen. Du solltest wissen, was Deployment, Service und ServiceAccount sind, und `kubectl` flüssig bedienen. Vorwissen zu Service Meshes brauchst du nicht.

So arbeitest du mit diesem Buch

Die Kapitel bauen aufeinander auf. Alle Manifeste liegen im Ordner `labs/` des Begleit-Repos github.com/thomaszachmann/linkerd-zum-anfassen. Klone es mit `git clone https://github.com/thomaszachmann/linkerd-zum-anfassen.git`. Die Befehle gehen davon aus, dass du in diesem Ordner stehst. Drei Kästen führen dich durch die Übungen:

Prüfpunkt

So kannst du feststellen, ob ein Schritt funktioniert hat.

Deine Aufgabe

Hier bist du dran. Versuche es zuerst selbst, bevor du weiterliest.

Achtung

Hier gibt es Fallstricke, an denen man wirklich hängen bleibt.

Was du über den Prüfstand wissen musst

Praxisbücher leben davon, dass ihre Befehle funktionieren. Deshalb stehen hier die Tatsachen zu dieser Auflage.

> **Hinweis**
>
> Die Labs liefen am 6. November 2026 mit `edge-26.11.1` in einer VM mit Ubuntu 26.04 auf arm64, mit 4 vCPUs, 8 GB RAM und 20 GB Platte. Prüfpunkte nennen dort, wo es zählt, die tatsächlich gemessenen Werte. Der Lauf hat mehrere Fehler der ersten Fassung aufgedeckt, die jetzt korrigiert sind.

Nicht alles ist abgedeckt: Der Multicluster-Lauf hat einen offenen Punkt, und einige Wege wurden gar nicht ausgeführt. Anhang B listet das genau auf. Wenn etwas bei dir anders aussieht, ist das ein Hinweis und nicht zwingend dein Fehler. Linkerd bewegt sich schnell, und die Edge-Releases erscheinen fast wöchentlich.

Versionen

Komponente	Version in diesem Buch	Hinweis
Linkerd	`edge-26.11.1` vom 2. November 2026	Linie 2.20, angekündigt am 23. Juni 2026
Kubernetes	1.35	Laut Doku für 2.20: 1.31 bis 1.35
Gateway API	1.5.1, Standard-Kanal	Laut Doku für 2.20: 1.2.1 bis 1.5.1
kind	0.33	Node-Image auf 1.35.8 gepinnt

Warum Edge? Das Open-Source-Projekt veröffentlicht seit Februar 2024 keine eigenen Stable-Artefakte mehr. Stabile Distributionen kommen von Anbietern wie Buoyant. Was frei verfügbar ist, sind Edge-Releases, und die stuft das Projekt selbst als grundsätzlich produktionsreif ein. Kapitel 14 geht auf die Folgen für den Betrieb ein.

Thomas Zachmann, November 2026

1 Warum ein Service Mesh?

Das lernst du hier: welches Problem ein Service Mesh löst, wie Linkerd sich einordnet und wann du besser keins einsetzt.

1.1 Das Problem steckt zwischen den Diensten

Sobald eine Anwendung aus mehreren Diensten besteht, wird das Netzwerk zum Teil der Anwendung. Plötzlich stellen sich Fragen, die keine Fachlogik sind und trotzdem jeder Dienst beantworten muss:

- Ist die Verbindung zwischen zwei Pods verschlüsselt, und weiß der Empfänger sicher, wer anruft?
- Welcher Dienst ist gerade langsam, und für wen?
- Was passiert, wenn ein Pod sporadisch Fehler liefert?
- Wie schicke ich zehn Prozent des Traffics an die neue Version?

Kubernetes beantwortet davon wenig. Ein Service verteilt Verbindungen, sieht aber keine einzelnen Requests. Eine NetworkPolicy arbeitet mit IP-Adressen und Ports, nicht mit Identitäten. Klassisch landet all das in Bibliotheken, und damit in jeder Sprache einzeln, in jedem Team neu und in jedem Upgrade als Aufwand.

1.2 Die Idee: ein Proxy neben jedem Pod

Ein Service Mesh zieht diese Aufgaben aus der Anwendung heraus. Neben jede Instanz kommt ein kleiner Proxy, der den gesamten TCP-Verkehr des Pods übernimmt. Die Anwendung spricht weiter unverschlüsseltes HTTP, gRPC oder TCP, und merkt von all dem nichts. Die Proxies bilden die **Data Plane**. Eine **Control Plane** versorgt sie mit dem, was sie wissen müssen: wohin ein Request gehen darf, welche Identität das Ziel haben muss, welche Regeln gelten.

Der Reiz liegt darin, dass die Fähigkeiten einheitlich für alle Dienste gelten, unabhängig von Sprache und Framework, und ohne Änderung am Anwendungscode.

1.3 Was Linkerd anders macht

Linkerd ist ein Projekt der Cloud Native Computing Foundation. Zwei Entwurfsentscheidungen prägen es.

Ein eigener Proxy. Linkerd setzt nicht auf einen Allzweck-Proxy. `linkerd2-proxy` ist in Rust geschrieben und ausdrücklich nur für den Einsatz im Service Mesh gedacht. Das Projekt nennt ihn deshalb einen *Micro-Proxy*.

Funktionieren ohne Konfiguration. Viele Funktionen laufen, sobald ein Pod im Mesh ist: mTLS zwischen gemeshten Pods, Metriken, latenzbasiertes Load Balancing. Du konfigurierst nur das, was vom Standard abweichen soll.

Das hat seinen Preis. Linkerd bietet nicht jeden Schalter, den ein Allzweck-Proxy mitbringt, und es ist kein Ingress-Controller. Wer sehr spezielle Layer-7-Filter braucht, stößt eher an Grenzen.

1.4 Wann du kein Service Mesh brauchst

Ein Mesh ist zusätzliche Infrastruktur, die du betreiben, aktualisieren und verstehen musst. Prüfe vor der Einführung ehrlich, ob es sich lohnt.

Situation	Einschätzung
Wenige Dienste, ein Team, keine mTLS-Pflicht	Vermutlich zu viel Aufwand
Compliance verlangt Verschlüsselung im Cluster	Starker Grund dafür
Viele Teams und Sprachen, uneinheitliche Metriken	Starker Grund dafür
Nur ein Canary-Rollout gewünscht	Oft reicht ein Rollout-Werkzeug am Ingress
Hauptsächlich Datenbankverkehr über TCP	Nur mTLS und Byte-Metriken, kein Request-Einblick

Diese Tabelle ist eine Empfehlung des Autors und keine Regel.

Deine Aufgabe

Nimm einen Cluster aus deinem Alltag. Schreib in drei Sätzen auf, welches konkrete Problem ein Mesh dort lösen würde. Fällt dir keines ein, ist das ebenfalls ein Ergebnis.

1.5 Zusammenfassung

Ein Service Mesh verlagert Sicherheit, Beobachtbarkeit und Zuverlässigkeit aus der Anwendung in eine einheitliche Proxy-Schicht. Linkerd setzt dabei auf einen spezialisierten Proxy und sinnvolle Standardwerte. Im nächsten Kapitel baust du dein Lab.

2 Das Lab aufbauen

Das lernst du hier: einen lokalen Cluster aufsetzen, die Linkerd-CLI installieren und die Control Plane starten.

Voraussetzungen: Docker oder Colima mit 4 CPUs und 8 GB RAM, dazu `kubectl`, `kind`, `helm` und `jq`.

2.1 Der Cluster

Ein Detail entscheidet hier über Erfolg oder Frust: kind v0.33 startet standardmäßig Kubernetes 1.37. Für Linkerd 2.20 nennt die Doku aber nur Versionen bis 1.35. Deshalb pinnt die Konfiguration das Node-Image.

```
# Lab-Cluster für "Linkerd zum Anfassen".
# Der Node ist bewusst festgepinnt: kind v0.33 startet sonst
Kubernetes 1.37,
# Linkerd 2.20 ist laut Doku aber nur bis Kubernetes 1.35
freigegeben.
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: linkerd-lab
nodes:
  - role: control-plane
    image: kindest/
node:v1.35.8@sha256:07b2536e30b803ed61d1677a79df6115f798ce64c80f9e22f6ed45afd093
  - role: worker
    image: kindest/
node:v1.35.8@sha256:07b2536e30b803ed61d1677a79df6115f798ce64c80f9e22f6ed45afd093
  - role: worker
    image: kindest/
node:v1.35.8@sha256:07b2536e30b803ed61d1677a79df6115f798ce64c80f9e22f6ed45afd093
```

```
kind create cluster --config labs/00-cluster/kind-config.yaml
kubectl wait --for=condition=Ready nodes --all --timeout=120s
kubectl get nodes
```

Das `wait` ist nötig: Direkt nach `kind create` stehen die Nodes noch einige Sekunden auf `NotReady`.

Prüfpunkt

Drei Nodes stehen auf `Ready`, und `kubectl version` meldet für den Server `v1.35.8`.

2.2 Die CLI

Die CLI läuft auf deinem Rechner und ist das wichtigste Werkzeug im Alltag. Sie erzeugt Manifeste, prüft den Cluster und zeigt den Zustand des Meshes. Das Projekt bietet ein Installationsskript an:

```
export LINKERD2_VERSION=edge-26.11.1
curl --proto '=https' --tlsv1.2 -sSfL https://run.linkerd.io/
install-edge | sh
export PATH=$HOME/.linkerd2/bin:$PATH
linkerd version --client
```

Ohne `LINKERD2_VERSION` bekommst du die jeweils neueste Edge-Version. Für ein reproduzierbares Lab pinnst du besser.

Hinweis

Wer ungern `curl | sh` ausführt, lädt die Binärdatei direkt von der GitHub-Release-Seite. Für `edge-26.11.1` heißen die Dateien zum Beispiel `linkerd2-cli-edge-26.11.1-darwin-arm64` oder `linkerd2-cli-edge-26.11.1-linux-amd64`. Das Release enthält keine `.sha256`-Datei, GitHub nennt die Prüfsumme aber pro Datei: `gh api repos/linkerd/linkerd2/releases/tags/edge-26.11.1 --jq '.assets[] | "\(.name) \(.digest)"'`. Vergleiche sie mit `sha256sum`, mache die Datei ausführbar und lege sie in den `PATH`.

2.3 Gateway API zuerst

Linkerd konfiguriert Routen über die `HTTPRoute`- und `GRPCRoute`-Typen der Kubernetes Gateway API. Diese CRDs bringt es standardmäßig nicht mit: Im Chart `linkerd-crds` steht `installGatewayAPI: false`. Du installierst sie also selbst, und zwar vor Linkerd.

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/
releases/download/v1.5.1/standard-install.yaml
```

```
kubectl get crds/httproutes.gateway.networking.k8s.io \
  -o "jsonpath={.metadata.annotations.gateway\.networking\.k8s\.io/
bundle-version}"
```

Prüfpunkt

Der zweite Befehl gibt `v1.5.1` aus. Meldet er `Not Found`, fehlt die Gateway API.

2.4 Die Control Plane

Linkerd wird in zwei Schritten installiert, erst die CRDs, dann die Control Plane. `linkerd install` wendet dabei nichts selbst an, sondern schreibt Manifeste auf die Standardausgabe. Das macht die Installation nachvollziehbar: Du kannst die Ausgabe auch in eine Datei lenken und lesen.

```
linkerd check --pre
linkerd install --crds | kubectl apply -f -
linkerd install | kubectl apply -f -
linkerd check
```

`linkerd check --pre` prüft, ob der Cluster bereit ist. `linkerd check` wartet, bis die Control Plane läuft, und untersucht sie gründlich.

Prüfpunkt

`linkerd check` endet ohne Fehler, und `kubectl -n linkerd get pods` zeigt laufende Pods für `linkerd-destination`, `linkerd-identity` und `linkerd-proxy-injector`.

Achtung

`linkerd install` ohne weitere Angaben erzeugt selbstsignierte Zertifikate mit einem Jahr Laufzeit. Für das Lab ist das in Ordnung, für die Produktion nicht. Kapitel 7 zeigt den richtigen Weg.

2.5 Die Viz-Erweiterung

Metriken, `tap` und das Dashboard liefert die Erweiterung Viz. Sie läuft als eigener Namespace `linkerd-viz`.

```
linkerd viz install | kubectl apply -f -
linkerd viz check
```

2.6 Aufräumen

Das Lab bleibt für die folgenden Kapitel bestehen. Wenn du fertig bist, entfernt dieser Befehl alles:

```
kind delete cluster --name linkerd-lab
```

2.7 Zusammenfassung

Du hast einen Cluster mit passender Kubernetes-Version, die Gateway-API-CRDs, die Control Plane und Viz. Merke dir die Reihenfolge: Gateway API, Linkerd-CRDs, Control Plane, Erweiterungen.

3 Wie Linkerd aufgebaut ist

Das lernst du hier: was die Komponenten tun, wie der Proxy in den Pod kommt und welcher Proxy für welche Aufgabe zuständig ist.

3.1 Das Gesamtbild

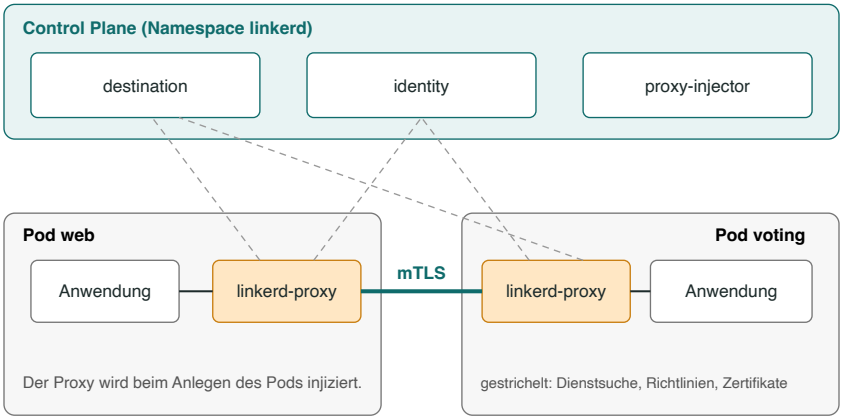


Abbildung 1: Control Plane und Data Plane: Die Proxies holen sich Dienstinformationen, Richtlinien und Zertifikate von der Control Plane und sprechen untereinander mTLS.

3.2 Control Plane

Die Control Plane läuft im Namespace `linkerd` und besteht aus drei Kernkomponenten.

destination. Hierher fragen die Proxies, wohin ein Request gehen soll und welche TLS-Identität sie am anderen Ende erwarten müssen. Auch die Richtlinien kommen von hier.

identity. Eine Zertifizierungsstelle. Sie nimmt Zertifikatsanfragen (CSRs) der Proxies an und signiert sie.

proxy-injector. Ein Admission Webhook. Er wird bei jedem neuen Pod aufgerufen und sucht die Annotation `linkerd.io/inject: enabled`. Ist sie

vorhanden, ändert er die Pod-Spezifikation und fügt die Linkerd-Container hinzu.

Hinweis

Der Injector wirkt nur beim Anlegen eines Pods. Annotierst du einen Namespace, passiert mit laufenden Pods nichts. Sie müssen neu starten.

3.3 Data Plane

In jedem gemeshten Pod läuft der Container `linkerd-proxy`. Damit der Verkehr ihn überhaupt erreicht, richtet der Init-Container `linkerd-init` iptables-Regeln im Pod ein. Alternativ erledigt das Linkerds CNI-Plugin, was hilfreich ist, wenn Pods keine erhöhten Netzwerkrechte bekommen dürfen.

Seit Linkerd 2.20 wird der Proxy standardmäßig als *Native Sidecar* gestartet. Das ist die Kubernetes-Funktion für Sidecars: Ein Init-Container mit `restartPolicy: Always`, der vor den Anwendungscontainern startet und über ihre Laufzeit hinaus weiterläuft. Damit entfallen laut Projekt typische Sidecar-Probleme, etwa bei Jobs und beim Container-Start. Im Chart steht das als `proxy.nativeSidecar: true`.

Achtung

Durch Native Sidecars sieht der Proxy in `kubectl get pod -o yaml` anders aus als in älteren Anleitungen. Er steht unter `initContainers`, nicht unter `containers`. Das betrifft auch eigene Skripte, die nach dem Container suchen.

Prüfpunkt

Sobald in Kapitel 4 Pods im Mesh sind, zeigt dir dieser Befehl den Unterschied:

```
kubectl -n emojivoto get pod -l app=web-svc -o jsonpath='{.items[0].spec.initContainers[*].name}'
```

3.4 Outbound und Inbound

Jeder Request in einer gemeshten Verbindung läuft durch zwei Proxies, und sie haben verschiedene Aufgaben. Das ist der wichtigste Satz dieses Kapitels, denn er erklärt später fast jedes Verhalten.

Proxy	Sitzt bei	Zuständig für
Outbound	Client	Dienstsuche, Load Balancing, Circuit Breaker, Retries, Timeouts
Inbound	Server	Durchsetzung der Autorisierung

Daraus folgen zwei Dinge, die du dir merken solltest:

1. Retries und Timeouts wirken nur, wenn der **Client** gemesht ist. Ein ungemeshter Client hat keinen Outbound-Proxy, der sie ausführen könnte.
2. Autorisierung wirkt auch gegen ungemeshte Clients, denn der **Server**-Proxy setzt sie durch.

3.5 Protokollerkennung

Der Proxy schaut sich die ersten Bytes einer Verbindung an, um HTTP zu erkennen. Klappt das, bekommst du Metriken und Routing auf Request-Ebene. Klappt es nicht, behandelt er die Verbindung als undurchsichtiges TCP. mTLS und Byte-Metriken funktionieren weiterhin.

Ruft die Anwendung selbst mit TLS auf, kann Linkerd nicht hineinsehen und behandelt die Verbindung ebenfalls als TCP. Protokolle, bei denen der Server zuerst spricht, sind ein Sonderfall. Dazu mehr in Kapitel 15.

Deine Aufgabe

Ein ungemeshter Pod ruft einen gemeshten Service, auf dem eine Retry-Regel liegt. Werden fehlgeschlagene Requests wiederholt? Begründe es mit der Tabelle oben, bevor du weiterliest.

Die Antwort lautet nein. Retries sind Sache des Outbound-Proxys, und den hat dieser Client nicht.

3.6 Zusammenfassung

Die Control Plane liefert Wissen, die Data Plane erledigt die Arbeit. Der Client-Proxy kümmert sich um Zuverlässigkeit, der Server-Proxy um Autorisierung.

4 Die erste Anwendung im Mesh

Das lernst du hier: eine Anwendung ins Mesh nehmen, ohne ihren Code anzufassen, und den Effekt sichtbar machen.

4.1 Emojivoto

Emojivoto ist die Demo-Anwendung des Linkerd-Projekts, eine kleine Abstimmung über Emojis. Sie besteht aus den Diensten `web`, `emoji` und `voting` sowie dem Lastgenerator `vote-bot`. `web` spricht HTTP nach außen und gRPC nach innen. Die Anwendung enthält einen absichtlichen Fehler, den du später findest.

Zuerst läuft sie ohne Mesh:

```
curl --proto '=https' --tlsv1.2 -sSfL https://run.linkerd.io/
emojivoto.yml \
  | kubectl apply -f -
kubectl -n emojivoto rollout status deploy/web
kubectl -n emojivoto port-forward svc/web-svc 8080:80
```

Öffne `http://localhost:8080` und schau dich um. Beende den Port-Forward anschließend wieder.

4.2 Ins Mesh nehmen

`linkerd inject` fügt dem Manifest nur die Annotation `linkerd.io/inject: enabled` hinzu. Den Rest erledigt der Injector im Cluster, sobald die Pods neu entstehen.

```
kubectl get -n emojivoto deploy -o yaml \
  | linkerd inject - \
  | kubectl apply -f -

linkerd check -n emojivoto --proxy -o short
```

Prüfpunkt

`linkerd check --proxy` läuft ohne Fehler durch, und das Kommando aus Kapitel 3 zeigt `linkerd-init` und `linkerd-proxy` als Init-Container.

Du hättest dasselbe erreichen können, indem du die Annotation direkt in dein Manifest schreibst. Das ist in GitOps-Umgebungen der übliche Weg, oft auf Ebene des Namespace. Die späteren Labs machen es so.

```
apiVersion: v1
kind: Namespace
metadata:
  name: resilienz
  annotations:
    linkerd.io/inject: enabled
```

4.3 Der erste Blick

Ohne eine einzige Zeile Konfiguration hast du jetzt Metriken pro Dienst.

```
linkerd viz -n emojivoto stat deploy
linkerd viz dashboard
```

Prüfpunkt

`stat` zeigt alle vier Deployments als gemesht mit `1/1`. Im Testlauf lag `emoji` bei 100 Prozent, `web` bei knapp 92 und `voting` bei knapp 86 Prozent.

4.4 Den Fehler finden

Die Erfolgsquote von `web` ist nicht perfekt. Laut Linkerd-Doku sind zwei Aufrufe betroffen: der Aufruf von `vote-bot` an `/api/vote` und der Aufruf `VoteDoughnut` von `web` an `voting`. Finde das selbst heraus, ohne ins Log zu schauen.

```
linkerd viz -n emojivoto tap deploy/web
linkerd viz -n emojivoto top deploy/web
```

`tap` zeigt die Requests live. `top` fasst sie nach Pfad zusammen und braucht dafür ein echtes Terminal. In Skripten oder über eine Pipe bricht es mit `open /dev/tty` ab.

Deine Aufgabe

Finde mit `top` heraus, welcher Pfad fehlschlägt. Was sagt dir das über die Reihenfolge, in der `web` und `voting` den Fehler sehen?

Das passt zusammen: Der eingehende `/api/vote`-Aufruf scheitert, weil der ausgehende `VoteDoughnut`-Aufruf scheitert. Du hast die Fehlerquelle eingegrenzt, ohne dass die Anwendung dafür etwas ausgibt.

4.5 Zusammenfassung

Eine Annotation und ein Neustart der Pods genügen, um eine Anwendung ins Mesh zu nehmen. Metriken und Live-Einblick gibt es sofort, mTLS ebenfalls, und darum geht es im nächsten Kapitel.

5 mTLS und Identität

Das lernst du hier: was beim automatischen mTLS passiert, woher die Identität eines Pods kommt und wie du die Verschlüsselung nachweist.

5.1 Identität statt IP-Adresse

Bei mTLS weisen sich beide Seiten mit einem Zertifikat aus. Linkerd bindet dieses Zertifikat an den **ServiceAccount** des Pods. Die Identität hat die Form:

```
<serviceaccount>.<namespace>.serviceaccount.identity.linkerd.cluster.local
```

Das ist der Grund, warum in Emojivoto jeder Dienst einen eigenen ServiceAccount hat. Teilen sich zwei Dienste den ServiceAccount `default`, kann das Mesh sie nicht unterscheiden.

5.2 Was beim Start eines Proxys passiert

1. Der Proxy erhält den **Trust Anchor** über eine Umgebungsvariable.
2. Er erzeugt einen privaten Schlüssel. Der liegt in einem `tmpfs-emptyDir`, bleibt also im Speicher und verlässt den Pod nicht.
3. Er sendet einen CSR an `identity`, zusammen mit dem ServiceAccount-Token des Pods.
4. `identity` prüft das Token und signiert das Zertifikat.
5. Das Zertifikat gilt 24 Stunden und wird automatisch erneuert.

Baut der Proxy nun eine Verbindung auf, kennt er aus der Antwort von `destination` die erwartete Identität des Ziels und prüft sie beim TLS-Handshake.

Laut Doku verwendet Linkerd für gemeshte Verbindungen TLS 1.3, einen hybriden Schlüsselaustausch mit ML-KEM-768 und X25519 sowie die Ciphersuite `AES_128_GCM`.

5.3 Den Nachweis führen

Vertrauen ist gut, gerade bei Sicherheitsthemen aber kein Beleg. Drei Wege zeigen dir, was wirklich passiert.

Der Überblick. `edges` listet die Verbindungen und ob sie gesichert sind:

```
linkerd viz -n emojivoto edges deployment
```

Live pro Request. `tap` zeigt das Feld `tls`:

```
linkerd viz -n emojivoto tap deploy/voting | head -20
```

Steht dort `tls=true`, ist der Request gesichert. Bei Quellen ohne Proxy steht dort `tls=no_tls_from_remote`. In Emojivoto siehst du das zunächst nicht, denn alle Dienste sind gemesht und es gibt keine Probes. Am Ende des Kapitels erzeugst du den Fall selbst.

Auf dem Draht. Der Debug-Sidecar enthält `tshark` und zeigt dir, was tatsächlich über das Netz läuft:

```
curl --proto '=https' --tlsv1.2 -sSfL https://run.linkerd.io/
emojivoto.yml \
  | linkerd inject --enable-debug-sidecar - \
  | kubectl apply -f -

kubectl -n emojivoto rollout status deploy/voting
kubectl -n emojivoto exec -it deploy/voting -c linkerd-debug -- /
bin/bash

# Im Container: der Verkehr zwischen den Pods
tshark -i any -f 'tcp port 4143'
```

Entscheidend ist der Port. Der Client-Proxy spricht nicht Port 8080 des Ziels an, sondern den Inbound-Port 4143 des Proxys. Erst der entschlüsselt und reicht den Request innerhalb des Pods an die Anwendung auf Port 8080 weiter.

Prüfpunkt

Der Verkehr von `web` zu `voting` läuft über Port 4143 und erscheint als `Application Data`. `tshark` schreibt dabei `TLSv1.2`, obwohl die Verbindung TLS 1.3 nutzt: Die Versionskennung im Record-Header ist aus Kompatibilitätsgründen festgelegt, und ohne den Handshake im Mitschnitt sieht `tshark` nur diese.

Das Gegenstück zeigt, was der Proxy dir abnimmt. Auf Port 8080 läuft der Weg zwischen Proxy und Anwendung im selben Pod, und der ist Klartext:

```
tshark -i any -f 'tcp port 8080' -d tcp.port==8080,http2 \
  -T fields -e http2.headers.path
```

Prüfpunkt

Hier stehen die Pfade lesbar da, zum Beispiel `/emojivoto.v1.VotingService/VotePoop`. Das ist kein Leck, denn dieser Verkehr verlässt den Pod nicht. Es erklärt aber, warum ein Mitschnitt auf dem falschen Port in die Irre führt.

Der Debug-Sidecar gehört nicht in den Dauerbetrieb. Wende das Original-Manifest ohne das Flag erneut an. Die Pipeline aus Kapitel 4 reicht dafür nicht, denn `kubectl get deploy` würde die Debug-Annotation einfach mitnehmen.

```
curl --proto '=https' --tlsv1.2 -sSfL https://run.linkerd.io/
emojivoto.yml \
  | linkerd inject - \
  | kubectl apply -f -
```

5.4 Die wichtigste Einschränkung

Achtung

mTLS ist nur zwischen gemeshten Pods Pflicht. Standardmäßig nimmt Linkerd weiterhin Klartext von ungemeshten Quellen an. Allein durch das Mesh ist dein Dienst also nicht abgeschottet. Das ändert erst die Autorisierung im nächsten Kapitel.

Das lässt sich direkt ausprobieren:

```
kubectl run ungemesht --rm -it --image=curlimages/curl:8.23.0 --
restart=Never \
  -- curl -s -o /dev/null -w '%{http_code}\n' http://web-svc.
emojivoto/
```

Deine Aufgabe

Sage voraus, was dieser Aufruf ergibt. Der Pod läuft im Namespace `default` ohne Proxy. Wird der Aufruf abgelehnt? Prüfe anschließend mit `tap` auf `deploy/web`, wie der Request dort erscheint.

5.5 Zusammenfassung

Jeder Proxy hat eine an den ServiceAccount gebundene, kurzlebige Identität. Gemeshter Verkehr ist verschlüsselt und gegenseitig authentifiziert. Ungemeshter Verkehr ist standardmäßig weiter erlaubt.

6 Autorisierung

Das lernst du hier: Zugriffe einschränken, ohne dabei die Produktion lahmzulegen, und das pro Dienst oder pro Route.

6.1 Das Entscheidungsmodell

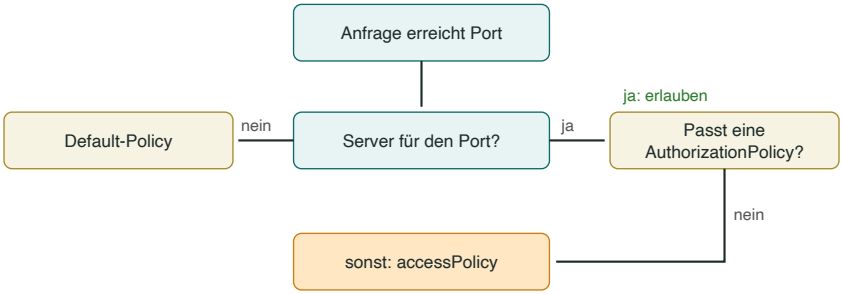


Abbildung 2: So entscheidet der Inbound-Proxy: Ohne Server gilt die Default-Policy. Mit Server braucht der Client eine passende Autorisierung, sonst greift die accessPolicy.

Vier Ressourcen wirken zusammen:

Ressource	Frage	API-Version
`Server`	Welcher Port welcher Pods?	`policy.linkerd.io/v1beta3`
`HTTPRoute`	Welcher Teil des Verkehrs?	`gateway.networking.k8s.io/v1`
`MeshTLSAuthentication`	Welche Identitäten?	`policy.linkerd.io/v1alpha1`
`AuthorizationPolicy`	Wer darf was?	`policy.linkerd.io/v1alpha1`

Für IP-Bereiche gibt es zusätzlich `NetworkAuthentication`.

Achtung

Viele Beispiele, auch in der Linkerd-Doku, zeigen noch `Server` in `v1beta1`. Diese Version ist in den CRDs von `edge-26.11.1` als deprecated markiert, aktuell ist `v1beta3`. Auch `ServerAuthorization` aus älteren Anleitungen brauchst du nicht. Nutze `AuthorizationPolicy`.

6.2 Die Default-Policy

Hat ein Port keinen `Server`, gilt die Default-Policy. Sie steht auf `all-unauthenticated`, was alles erlaubt.

Wert	Bedeutung
`all-unauthenticated`	Alles erlaubt (Standard)
`all-authenticated`	Gemeshte Clients, auch aus anderen Clustern
`cluster-authenticated`	Gemeshte Clients aus diesem Cluster
`cluster-unauthenticated`	Gemeshte und ungemeshte Clients aus diesem Cluster
`deny`	Alles blockiert
`audit`	Wie `all-unauthenticated`, markiert Requests aber in Logs und Metriken

Überschreiben lässt sich das pro Namespace oder Workload mit der Annotation `config.linkerd.io/default-inbound-policy`.

Achtung

Das Feld `accessPolicy` eines `Server` hat den Standardwert `deny`. Sobald du einen `Server` anlegst, werden alle Clients ohne passende Autorisierung abgelehnt, und zwar sofort. In einer laufenden Umgebung ist das der klassische Weg in den Ausfall.

Linkerd autorisiert auf Ports ohne `Server` automatisch die Health- und Readiness-Probes. Legst du einen `Server` für einen Port an, auf dem Probes laufen, musst du sie selbst erlauben. Emojivoto hat keine Probes, deine Anwendungen aber sehr wahrscheinlich schon.

6.3 Lab: den Voting-Dienst abschotten

Nur `web` soll `voting` aufrufen dürfen. Du gehst dreistufig vor, so wie du es auch in der Produktion tun solltest. Zuerst brauchst du einen Test-Client mit eigener Identität.

```
# Ein gemeshter Test-Client mit eigener Identität (ServiceAccount
lab-client).
apiVersion: v1
kind: ServiceAccount
metadata:
  name: lab-client
  namespace: emojivoto
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab-client
  namespace: emojivoto
spec:
  replicas: 1
  selector:
    matchLabels:
      app: lab-client
  template:
    metadata:
      labels:
        app: lab-client
      annotations:
        linkerd.io/inject: enabled
    spec:
      serviceAccountName: lab-client
      containers:
        - name: curl
          image: curlimages/curl:8.23.0
          command: ["sh", "-c", "sleep infinity"]
```

```
kubectl apply -f labs/00-cluster/lab-client.yaml
kubectl -n emojivoto rollout status deploy/lab-client
```

6.3.1 Schritt 1: Beobachten

Der `Server` geht im Audit-Modus live. Es wird nichts blockiert, Verstöße werden aber markiert.

```
# Schritt 1: Server im Audit-Modus. Nichts wird blockiert, Verstöße
werden nur markiert.
apiVersion: policy.linkerd.io/v1beta3
kind: Server
metadata:
  name: voting-grpc
  namespace: emojivoto
spec:
  podSelector:
    matchLabels:
      app: voting-svc
  port: grpc
  proxyProtocol: gRPC
  accessPolicy: audit
```

```
kubectl apply -f labs/06-autorisierung/01-server-audit.yaml
linkerd viz authz -n emojivoto deploy/voting
```

Die Doku beschreibt, dass der Proxy in diesem Modus Log-Einträge mit `authz.name=audit` schreibt und Metriken wie `request_total` das Label `authz_name=audit` tragen. Dort siehst du, wer ohne Regel durchgelassen wurde.

6.3.2 Schritt 2: Regeln anlegen

```
# Schritt 2: Nur die Identität des Web-Frontends darf den Voting-
Dienst rufen.
apiVersion: policy.linkerd.io/v1alpha1
kind: MeshTLSAuthentication
metadata:
  name: voting-clients
  namespace: emojivoto
spec:
  identityRefs:
    - kind: ServiceAccount
      name: web
---
apiVersion: policy.linkerd.io/v1alpha1
kind: AuthorizationPolicy
metadata:
  name: voting-nur-web
  namespace: emojivoto
spec:
```

```
  targetRef:
    group: policy.linkerd.io
    kind: Server
    name: voting-grpc
  requiredAuthenticationRefs:
    - group: policy.linkerd.io
      kind: MeshTLSAuthentication
      name: voting-clients
```

```
kubectl apply -f labs/06-autorisierung/02-voting-erlauben.yaml
linkerd viz authz -n emojivoto deploy/voting
```

Prüfpunkt

`authz` zeigt den Verkehr von `web` jetzt unter `authorizationpolicy/voting-nur-web` statt unter Audit. Wichtig: Warte, bis über das Zeitfenster der Abfrage kein Audit-Verkehr mehr von erwarteten Clients auftaucht.

6.3.3 Schritt 3: Scharf schalten

```
kubectl apply -f labs/06-autorisierung/03-server-erzwingen.yaml
```

Jetzt der Gegentest mit dem Client, der keine Erlaubnis hat. Weil `voting` gRPC spricht, simuliert `curl` einen gRPC-Aufruf:

```
kubectl -n emojivoto exec deploy/lab-client -c curl -- \
  curl -sv --http2-prior-knowledge -X POST \
  -H 'content-type: application/grpc' \
  http://voting-svc.emojivoto:8080/emojivoto.v1.VotingService/
VoteDog 2>&1 \
  | grep -i grpc-status
```

Prüfpunkt

Die Antwort trägt `grpc-status: 7`, das entspricht `PermissionDenied`, und `grpc-message` nennt `unauthorized request on route`. Der HTTP-Status ist dabei `200`, denn gRPC meldet Fehler im Header. Die App läuft weiter, denn `web` ist autorisiert.

6.4 Lab: Autorisierung pro Route

Eine `AuthorizationPolicy` kann statt eines `Server` auch eine `HTTPRoute` schützen. Dann kann ein Client lesen, ohne abstimmen zu dürfen. Die Route hat den `Server` als Parent. Das ist eine **Inbound-Route**, im Gegensatz zu den Outbound-Routen ab Kapitel 9, die einen `Service` als Parent haben.

```
# Autorisierung pro Route: /api/vote nur für vote-bot
(ServiceAccount default),
# alles andere für vote-bot und lab-client.
apiVersion: policy.linkerd.io/v1beta3
kind: Server
metadata:
  name: web-http
  namespace: emojivoto
spec:
  podSelector:
    matchLabels:
      app: web-svc
  port: http
  proxyProtocol: HTTP/1
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: web-vote
  namespace: emojivoto
spec:
  parentRefs:
    - group: policy.linkerd.io
      kind: Server
      name: web-http
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api/vote
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: web-lesen
  namespace: emojivoto
spec:
  parentRefs:
```

```
    - group: policy.linkerd.io
      kind: Server
      name: web-http
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api/list
---
apiVersion: policy.linkerd.io/v1alpha1
kind: MeshTLSAuthentication
metadata:
  name: nur-vote-bot
  namespace: emojivoto
spec:
  identityRefs:
    - kind: ServiceAccount
      name: default
---
apiVersion: policy.linkerd.io/v1alpha1
kind: MeshTLSAuthentication
metadata:
  name: leser
  namespace: emojivoto
spec:
  identityRefs:
    - kind: ServiceAccount
      name: default
    - kind: ServiceAccount
      name: lab-client
---
apiVersion: policy.linkerd.io/v1alpha1
kind: AuthorizationPolicy
metadata:
  name: web-vote-erlauben
  namespace: emojivoto
spec:
  targetRef:
    group: gateway.networking.k8s.io
    kind: HTTPRoute
    name: web-vote
  requiredAuthenticationRefs:
    - group: policy.linkerd.io
      kind: MeshTLSAuthentication
      name: nur-vote-bot
```

```
---
apiVersion: policy.linkerd.io/v1alpha1
kind: AuthorizationPolicy
metadata:
  name: web-lesen-erlauben
  namespace: emojivoto
spec:
  targetRef:
    group: gateway.networking.k8s.io
    kind: HTTPRoute
    name: web-lesen
  requiredAuthenticationRefs:
    - group: policy.linkerd.io
      kind: MeshTLSAuthentication
      name: leser
```

```
kubectl apply -f labs/06-autorisierung/04-web-pro-route.yaml

web() {
  kubectl -n emojivoto exec deploy/lab-client -c curl -- \
    curl -s -o /dev/null -w '%{http_code}\n' "http://web-svc.
emojivoto$1"
}
web /api/list
web '/api/vote?choice=:cat:'
```

Prüfpunkt

`/api/list` ergibt `200`, denn `lab-client` steht in `leser`. `/api/vote` ergibt `403`, denn dort ist nur `default` zugelassen. Andere Pfade wie `/` treffen auf keine Route und ergeben `404`. Die Metrik `inbound_http_route_not_found_total` zählt solche Requests.

Achtung

Sobald an einem `Server` Routen hängen, ist alles außerhalb dieser Routen nicht erreichbar. Wer Routen einführt, muss jeden genutzten Pfad abdecken, auch Probes und Metrik-Endpunkte.

Deine Aufgabe

`vote-bot` läuft mit dem ServiceAccount `default`. Erlaube `lab-client` den Pfad `/api/vote` ebenfalls, indem du nur eine Ressource änderst. Welche ist es, und warum wäre es falsch, stattdessen `lab-client` zum `default`-ServiceAccount zu machen?

6.5 Zusammenfassung

Führe Autorisierung immer in drei Schritten ein: Audit, Regeln, Durchsetzung. `Server` beschreiben den Port, `AuthorizationPolicy` verbindet Ziel und Identitäten. Probes musst du selbst bedenken, sobald ein `Server` im Spiel ist.

7 Zertifikate für den Ernstfall

Das lernst du hier: die Vertrauenskette, eine Installation mit eigenen Zertifikaten und wie du eine Rotation planst.

7.1 Die Vertrauenskette

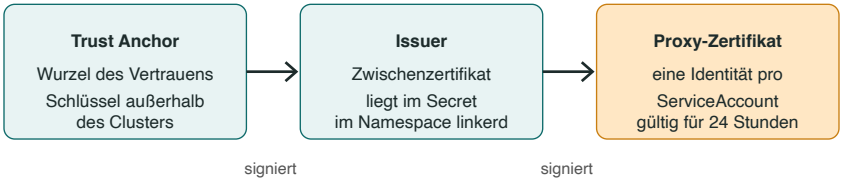


Abbildung 3: Vertrauenskette: Der Trust Anchor signiert den Issuer, der Issuer signiert die kurzlebigen Proxy-Zertifikate.

Die Proxy-Zertifikate rotieren von selbst. Um die beiden oberen Ebenen musst du dich kümmern.

Ebene	Wo liegt der Schlüssel?	Wer rotiert?
Trust Anchor	Am besten außerhalb des Clusters	Du, sorgfältig geplant
Issuer	Secret im Namespace `linkerd`	Du oder cert-manager
Proxy	`tmpfs` im Pod	Linkerd, Laufzeit 24 Stunden

Läuft der Issuer ab, können Proxies keine neuen Zertifikate mehr bekommen. Plane dafür Überwachung ein. Die Installation aus Kapitel 2 erzeugt Zertifikate mit einem Jahr Laufzeit, die nicht von selbst erneuert werden.

Hinweis

Multicluster setzt einen gemeinsamen Trust Anchor voraus. Die Standardinstallation funktioniert dafür nicht, denn jeder Cluster hätte seinen eigenen.

7.2 Lab: eigene Zertifikate mit step

Du brauchst step. Für dieses Lab legst du einen zweiten, frischen Cluster an. Getestet wurde das, nachdem der erste Cluster gelöscht war. Beide parallel kosten deutlich mehr Speicher. Erzeuge die Dateien außerhalb des Buchordners, denn `ca.key` ist hochsensibel und gehört weder in die Dropbox noch in ein Git-Repository.

```
kind create cluster --config labs/00-cluster/kind-config.yaml --
name linkerd-pki
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/
releases/download/v1.5.1/standard-install.yaml

mkdir -p ~/linkerd-pki-lab && cd ~/linkerd-pki-lab

# Trust Anchor
step certificate create root.linkerd.cluster.local ca.crt ca.key \
  --profile root-ca --no-password --insecure

# Issuer, ein Jahr gültig
step certificate create identity.linkerd.cluster.local issuer.crt
issuer.key \
  --profile intermediate-ca --not-after 8760h --no-password --
insecure \
  --ca ca.crt --ca-key ca.key
```

Achtung

`--no-password --insecure` lässt den Schlüssel unverschlüsselt. Linkerd braucht das für den Issuer, weil es ihn lesen muss. Für den Schlüssel des Trust Anchor ist das in echten Umgebungen nicht akzeptabel. Er gehört in ein Secret-Management-System oder auf ein Offline-Medium. Für einen langlebigeren Trust Anchor ergänzt du zum Beispiel `--not-after=87600h`.

Jetzt die Installation mit Helm, wie du sie auch produktiv verwenden würdest:

```
helm repo add linkerd-edge https://helm.linkerd.io/edge
helm repo update

helm install linkerd-crds linkerd-edge/linkerd-crds \
```

```
  -n linkerd --create-namespace --version 2026.11.1

helm install linkerd-control-plane -n linkerd \
  --set-file identityTrustAnchorsPEM=ca.crt \
  --set-file identity.issuer.tls.crtPEM=issuer.crt \
  --set-file identity.issuer.tls.keyPEM=issuer.key \
  --version 2026.11.1 \
  linkerd-edge/linkerd-control-plane

linkerd check
```

Die Chart-Version `2026.11.1` gehört zu `edge-26.11.1`. Ohne `--version` nimmt Helm die neueste Version. Pinne sie in der Produktion immer.

Die CLI kennt dafür ebenfalls Flags: `--identity-trust-anchors-file`, `--identity-issuer-certificate-file` und `--identity-issuer-key-file`.

Prüfpunkt

`linkerd check` läuft durch. Vergleiche außerdem die Fingerabdrücke: `step certificate fingerprint ca.crt` muss zum Trust Anchor im Cluster passen, den du mit `kubectl -n linkerd get cm linkerd-identity-trust-roots -o jsonpath='{.data.ca-bundle\.crt}' | step certificate fingerprint` ausliest.

Den Lab-Cluster kannst du danach wieder löschen. Das `kubectl`-Kontext-Setzen ist dabei wichtig: `kind delete` lässt den aktuellen Kontext leer, und `kubectl` versucht dann `localhost:8080` zu erreichen.

```
kind delete cluster --name linkerd-pki
kubectl config use-context kind-linkerd-lab
rm -rf ~/linkerd-pki-lab
```

7.3 Automatische Rotation mit cert-manager

Von Hand rotieren skaliert nicht. Die Doku empfiehlt für die Produktion cert-manager und trust-manager. Das Prinzip:

1. cert-manager erzeugt den Issuer aus dem Trust Anchor und erneuert ihn automatisch. Das Secret liegt im Namespace `linkerd`.
2. trust-manager schreibt das Trust-Bundle in eine ConfigMap im Namespace `linkerd`. Sie enthält keine privaten Schlüssel.

3. Linkerd wird so installiert, dass es diese externen Quellen nutzt. Bei der CLI steuert das Flag `--identity-external-issuer` diesen Weg.

Das Tutorial Automatically Rotating Control Plane TLS Credentials zeigt alle Manifeste.

Die Rotation des Issuers läuft damit von selbst. Die Rotation des Trust Anchor bleibt laut Doku auch dann mit Handarbeit verbunden: neuen Anchor ins Bundle aufnehmen, Control Plane und Data Plane neu starten, Issuer erneuern, wieder neu starten, alten Anchor aus dem Bundle entfernen.

Hinweis

Buoyant bewirbt für seine Distribution ab 2.20 eine automatisierte Trust-Anchor-Rotation. Die Ankündigung des Open-Source-Projekts zu 2.20 beschreibt das nicht. Geh also von Handarbeit aus, wenn du Edge-Releases einsetzt.

7.4 Zusammenfassung

Behandle den Trust Anchor wie eine Wurzel-CA und den Issuer als rotierendes Betriebsgeheimnis. In der Produktion gehören automatische Erneuerung und eine Warnung vor dem Ablauf zur Grundausstattung.

8 Beobachten

Das lernst du hier: mit welchen Werkzeugen du das Mesh befragst und wie die Metriken in dein Monitoring kommen.

8.1 Die goldenen Signale

Jeder Proxy misst für HTTP und gRPC Erfolgsquote, Requests pro Sekunde und Latenz. Mit diesen drei Werten beantwortest du die Frage, ob es einem Dienst gut geht.

8.2 Werkzeuge in der CLI

Befehl	Zeigt	Typische Frage
`linkerd viz stat`	Kennzahlen pro Ressource	Welcher Dienst leidet?
`linkerd viz stat-inbound`	Eingehender Verkehr	Wie geht es dem Dienst als Server?
`linkerd viz stat-outbound`	Ausgehender Verkehr	Wie erlebt der Client seine Ziele?
`linkerd viz top`	Aufrufe pro Pfad, live	Welcher Pfad ist heiß?
`linkerd viz tap`	Einzelne Requests, live	Was passiert genau?
`linkerd viz edges`	Verbindungen, mTLS-Status	Wer spricht mit wem?
`linkerd viz authz`	Treffer pro Autorisierung	Welche Regel greift?

```
linkerd viz -n emojivoto stat deploy
linkerd viz -n emojivoto stat-inbound deploy/web
linkerd viz -n emojivoto stat-outbound deploy/web
```

Deine Aufgabe

Vergleiche Inbound und Outbound von `deploy/web`. Du siehst dieselbe Anwendung aus zwei Richtungen. Welche Sicht zeigt den Fehler aus Kapitel 4 näher an der Ursache?

`stat` arbeitet laut Doku mit einem Zeitfenster von einer Minute. Änderungen werden also erst nach kurzer Zeit voll sichtbar. Das erklärt manche Verwirrung direkt nach einem `kubectl apply`.

Hinweis

`tap` gewährt Einblick in den laufenden Verkehr. Behandle den Zugriff darauf als sensibel. Die Doku widmet dem Thema die Seite „Securing Linkerd Tap“.

8.3 Rohe Metriken

Jeder Proxy stellt Prometheus-Metriken bereit. Am direktesten kommst du so an sie heran:

```
linkerd diagnostics proxy-metrics -n emojivoto deploy/web | less
```

Ein paar Namen, die du kennen solltest:

Metrik	Bedeutung
`request_total`, `response_total`	Requests und Antworten
`response_latency_ms`	Latenz-Histogramm
`inbound_http_authz_allow_total`	Durch Autorisierung erlaubt
`inbound_http_authz_deny_total`	Durch Autorisierung abgelehnt
`outbound_http_route_request_statuses_total`	Status pro Route
`outbound_http_route_backend_requests_total`	Requests pro Backend
`outbound_http_balancer_endpoints`	Endpunkte im Load Balancer
`control_identity_cert_expiration_timestamp_seconds`	Ablauf des Proxy-Zertifikats

Linkerd 2.20 hat außerdem die Inbound-Metriken stark ausgebaut. Die aktuelle Liste steht in der Referenz der Proxy-Metriken.

Achtung

Hat ein Pod einen strengen `Server` oder eine `deny`-Policy, kann auch Prometheus ausgesperrt sein. `linkerd viz check --proxy -n <ns>` weist darauf hin, und `linkerd viz allow-scrapes --namespace <ns> | kubectl apply -f -` erzeugt die nötigen Freigaben.

8.4 Prometheus in der Produktion

Viz bringt ein eigenes Prometheus mit. Für die Produktion empfiehlt die Doku eine eigene Instanz. Als Richtwert nennt sie etwa 5 MB Speicher pro gemeshtem Pod. Bei OOMKills helfen `storage.tsdb.retention.time` und `storage.tsdb.retention.size`.

Ein sinnvoller erster Alarm prüft die Zeit bis zum Ablauf der Zertifikate. Diese Abfrage lief im Lab gegen das Viz-Prometheus und lieferte dort knapp 84.000 Sekunden, also gut 23 Stunden. Als Alarm im Fehlerfall ausgelöst wurde sie nicht:

```
min(control_identity_cert_expiration_timestamp_seconds) - time() <
3600 * 6
```

Weil Proxy-Zertifikate 24 Stunden gelten und automatisch erneuert werden, sollte der Wert im Normalbetrieb über dieser Schwelle bleiben. Fällt er darunter, stimmt mit der Erneuerung etwas nicht.

8.5 Zusammenfassung

Starte mit `stat`, grenze mit `top` ein und sieh dir mit `tap` einzelne Requests an. Für den Dauerbetrieb gehören die Metriken in dein eigenes Prometheus.

9 Retries und Timeouts

Das lernst du hier: Fehler abfedern, ohne die Lage zu verschlimmern, und langsame Dienste begrenzen.

9.1 Der Aufbau

Du baust zwei absichtlich schlechte Dienste. `flaky` scheitert bei 30 Prozent der Requests, `slow` braucht 800 Millisekunden. Das Werkzeug `bb` stammt von Buoyant, `slow_cooker` erzeugt Last.

```
# Zwei absichtlich schlechte Backends: "flaky" scheitert bei 30 %
der Requests,
# "slow" braucht 800 ms. Jedes bekommt einen eigenen Lastgenerator.
apiVersion: v1
kind: Namespace
metadata:
  name: resilienz
  annotations:
    linkerd.io/inject: enabled
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flaky
  namespace: resilienz
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flaky
  template:
    metadata:
      labels:
        app: flaky
    spec:
      containers:
        - name: terminus
          image: buoyantio/bb:v0.0.6
          args: ["terminus", "--h1-server-port=8080", "--percent-
failure=30"]
```

```
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: flaky
  namespace: resilienz
spec:
  selector:
    app: flaky
  ports:
    - name: http
      port: 8080
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: slow
  namespace: resilienz
spec:
  replicas: 2
  selector:
    matchLabels:
      app: slow
  template:
    metadata:
      labels:
        app: slow
    spec:
      containers:
        - name: terminus
          image: buoyantio/bb:v0.0.6
          args: ["terminus", "--h1-server-port=8080", "--sleep-in-
millis=800"]
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: slow
  namespace: resilienz
spec:
```

```
  selector:
    app: slow
  ports:
    - name: http
      port: 8080
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: last-flaky
  namespace: resilienz
spec:
  replicas: 1
  selector:
    matchLabels:
      app: last-flaky
  template:
    metadata:
      labels:
        app: last-flaky
    spec:
      containers:
        - name: slow-cooker
          image: buoyantio/slow_cooker:1.3.0
          command: ["/bin/sh", "-c"]
          args:
            - |
              sleep 5
              /slow_cooker/slow_cooker --qps 10 http://flaky:8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: last-slow
  namespace: resilienz
spec:
  replicas: 1
  selector:
    matchLabels:
      app: last-slow
  template:
    metadata:
      labels:
        app: last-slow
```

```
    spec:
      containers:
        - name: slow-cooker
          image: buoyantio/slow_cooker:1.3.0
          command: ["/bin/sh", "-c"]
          args:
            - |
              sleep 5
              /slow_cooker/slow_cooker --qps 5 http://slow:8080
```

```
kubectl apply -f labs/09-resilienz/demo.yaml
kubectl -n resilienz rollout status deploy/flaky
linkerd viz -n resilienz stat-outbound deploy/last-flaky
```

Prüfpunkt

Warte eine gute Minute, bis Pods laufen und das Zeitfenster gefüllt ist. Dann zeigt `stat-outbound` rund 70 Prozent Erfolg, im Testlauf 68,95 Prozent. Das folgt direkt aus `--percent-failure=30`.

9.2 Retries

Retries konfigurierst du über Annotationen an einem `Service`, einer `HTTPRoute` oder einer `GRPCRoute`. Annotationen an einer Route ersetzen die am `Service` vollständig.

```
# Retries für alle Requests an den Service flaky.
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: flaky-retries
  namespace: resilienz
  annotations:
    retry.linkerd.io/http: 5xx
    retry.linkerd.io/limit: "2"
spec:
  parentRefs:
    - name: flaky
      kind: Service
      group: core
      port: 8080
  rules:
```

```
    - matches:
        - path:
            type: PathPrefix
            value: /
```

```
kubectl apply -f labs/09-resilienz/retry-route.yaml
linkerd viz -n resilienz stat-outbound deploy/last-flaky
```

Prüfpunkt

Ein Request scheitert nur, wenn der erste Versuch und beide Wiederholungen scheitern. Das sind 0,3 × 0,3 × 0,3 = 2,7 Prozent, also rund 97 Prozent Erfolg. Die Messung bestätigt das, braucht aber Geduld: 95,7 Prozent nach gut einer Minute, 96,5 nach zweieinhalb und 97,3 nach dreieinhalb Minuten. Die Spalte `RETRIES` steht bei rund 28 Prozent, und das Backend sieht etwa 14 statt 10 Requests pro Sekunde.

Annotation	Bedeutung
`retry.linkerd.io/http`	Codes: `504`, `500-504`, `gateway-error`, `5xx`
`retry.linkerd.io/grpc`	`cancelled`, `deadline-exceeded`, `internal`, `resource-exhausted`, `unavailable`
`retry.linkerd.io/limit`	Maximale Wiederholungen, Standard `1`
`retry.linkerd.io/timeout`	Abbruch und neuer Versuch nach dieser Zeit. Einheit ist Pflicht: `200ms`, `5s`

Requests mit einem Body über 64 KiB werden nicht wiederholt.

Achtung

Retries sind nur für Aufrufe sicher, die mehrfach ausgeführt werden dürfen. Eine Bestellung, deren Antwort verloren ging, ist vielleicht längst angelegt. Setze Retries gezielt auf Routen mit lesenden oder idempotenten Aufrufen, nicht pauschal auf den ganzen Dienst.

Retries vervielfachen außerdem die Last genau dann, wenn ein Dienst ohnehin Probleme hat. Halte `limit` klein.

Deine Aufgabe

Die Route in diesem Lab passt auf `/`. Baue daraus eine Route, die nur `GET`-Requests wiederholt. Tipp: `matches` kennt neben `path` auch `method`.

9.3 Timeouts

Timeouts funktionieren genauso über Annotationen:

```
# Harte Obergrenze: Nach 500 ms gibt der Client-Proxy auf.
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: slow-timeout
  namespace: resilienz
  annotations:
    timeout.linkerd.io/request: 500ms
spec:
  parentRefs:
    - name: slow
      kind: Service
      group: core
      port: 8080
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
```

```
kubectl apply -f labs/09-resilienz/timeout-route.yaml
linkerd viz -n resilienz stat-outbound deploy/last-slow
```

Prüfpunkt

Weil `slow` immer 800 ms braucht und das Limit bei 500 ms liegt, scheitern alle Requests: `SUCCESS` steht auf 0 Prozent, `TIMEOUTS` auf 100 Prozent. Der Client bekommt `504 Gateway Timeout`, und der Header `l5d-proxy-error` nennt die Ursache. Der Timeout funktioniert also, er ist nur absichtlich zu knapp. Setze ihn jetzt höher, dann steigt die Erfolgsquote auf 100 Prozent:

```
kubectl                                                    -n
resilienz annotate --overwrite httproutes.gateway.networking.k8s.io
slow-timeout timeout.linkerd.io/request=2s
```

Annotation	Begrenzt
`timeout.linkerd.io/request`	Den gesamten Request-Response-Stream
`timeout.linkerd.io/response`	Die Antwort des Backends
`timeout.linkerd.io/idle`	Leerlauf eines Streams

Auch das Feld `timeouts.request` einer `HTTPRoute` funktioniert. Sind Feld und Annotation gesetzt, gewinnt die Annotation.

Achtung

`kubectl get httproute` ist mehrdeutig, denn den Typ gibt es in den Gruppen `gateway.networking.k8s.io` und `policy.linkerd.io`. Schreibe die Ressource in Skripten immer aus, wie im Prüfpunkt oben.

9.4 Fallstricke

ServiceProfiles. Ältere Quellen warnen, dass ein `ServiceProfile` Retry-, Timeout- und Circuit-Breaker-Annotationen aushebelt. Im Test mit `edge-26.11.1` ließ sich das nicht zeigen: Die `HTTPRoute` aus diesem Lab blieb aktiv, sowohl mit einem leeren `ServiceProfile` als auch mit einem, das eine Route enthielt. Die Retries liefen weiter bei rund 27 Prozent. Getestet ist nur diese Kombination, nicht Annotationen direkt am `Service`. ServiceProfiles sind die ältere Konfiguration, mische sie also nicht ohne eigenen Test mit Routen.

Gemeshter Client. Der Outbound-Proxy führt Retries und Timeouts aus. Ein ungemeshter Client bekommt nichts davon.

Zusammenspiel. Die Referenz sagt, dass Timeouts dieser Art nicht zu Retries führen. Für wiederholbare Zeitüberschreitungen ist `retry.linkerd.io/timeout` da. Dieses Zusammenspiel wurde hier nicht getestet.

9.5 Zusammenfassung

Retries gehören auf idempotente Routen und bleiben klein. Timeouts setzen Grenzen. Beides lebt im Client-Proxy und wird über Annotationen gesteuert.

10 Circuit Breaking und Rate Limiting

Das lernst du hier: kaputte Endpunkte aussortieren und Dienste vor zu viel Last schützen.

10.1 Lab: Circuit Breaking

Hinter einem Service stehen ein guter und ein schlechter Pod. Der schlechte antwortet immer mit einem Fehler.

```
# Ein gesunder und ein kaputter Pod hinter demselben Service.
apiVersion: v1
kind: Namespace
metadata:
  name: circuit-breaking-demo
  annotations:
    linkerd.io/inject: enabled
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: good
  namespace: circuit-breaking-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      class: good
  template:
    metadata:
      labels:
        class: good
        app: bb
    spec:
      containers:
        - name: terminus
          image: buoyantio/bb:v0.0.6
          args: ["terminus", "--h1-server-port=8080"]
          ports:
            - containerPort: 8080
---
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bad
  namespace: circuit-breaking-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      class: bad
  template:
    metadata:
      labels:
        class: bad
        app: bb
    spec:
      containers:
        - name: terminus
          image: buoyantio/bb:v0.0.6
          args: ["terminus", "--h1-server-port=8080", "--percent-
failure=100"]
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: bb
  namespace: circuit-breaking-demo
spec:
  selector:
    app: bb
  ports:
    - name: http
      port: 8080
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: slow-cooker
  namespace: circuit-breaking-demo
spec:
  replicas: 1
  selector:
```

```
    matchLabels:
      app: slow-cooker
  template:
    metadata:
      labels:
        app: slow-cooker
    spec:
      containers:
        - name: slow-cooker
          image: buoyantio/slow_cooker:1.3.0
          command: ["/bin/sh", "-c"]
          args:
            - |
              sleep 5
              /slow_cooker/slow_cooker --qps 10 http://bb:8080
```

```
kubectl apply -f labs/10-circuit-breaker/demo.yaml
kubectl -n circuit-breaking-demo rollout status deploy/slow-cooker
linkerd viz -n circuit-breaking-demo stat deploy
linkerd viz -n circuit-breaking-demo stat deploy/slow-cooker --to
svc/bb
linkerd viz -n circuit-breaking-demo stat-outbound deploy/slow-
cooker
```

Prüfpunkt

Erwarte **nicht** 50 Prozent. Der Load Balancer verteilt nicht gleichmäßig, und die Quote schwankt von Lauf zu Lauf stark. In zwei Testläufen ohne Breaker lag die Erfolgsquote bei knapp 99 und bei 33 Prozent, und `slow-cooker` wechselte im Log zwischen `0/100` und `59/41` guten und schlechten Requests pro Intervall. Sieh dir deshalb an, wie viel Verkehr `bad` bekommt: Im zweiten Lauf waren es 7,6 von gut 10 Requests pro Sekunde.

Nun aktivierst du den Circuit Breaker mit einer Annotation am Service:

```
kubectl annotate -n circuit-breaking-demo svc/bb \
  balancer.linkerd.io/failure-accrual=consecutive

linkerd diagnostics policy -n circuit-breaking-demo svc/bb 8080 -o
```

```
json \
  | jq '.protocol.Kind.Detect.http1.failure_accrual'
```

Prüfpunkt

`diagnostics policy` zeigt `ConsecutiveFailures` mit `max_failures: 7`. Vor der Annotation steht dort `null`. Nach gut einer Minute lag `slow-cooker` im Test bei 99,83 Prozent Erfolg, und `bad` bekam noch 0,3 Requests pro Sekunde, `good` gut 10.

10.1.1 Wie das funktioniert

Der Breaker sitzt im Load Balancer des Outbound-Proxys und arbeitet pro Endpunkt. Nach sieben Fehlern in Folge markiert er den Pod als nicht verfügbar. Nach einer Mindeststrafe von einer Sekunde kommt der Pod in die *Probation*: Er darf genau einen Request beantworten. Klappt das, ist er zurück. Scheitert es, wächst die Wartezeit exponentiell bis maximal eine Minute.

Achtung

Der Test-Request der Probation ist ein echter Anwendungs-Request, keine Readiness-Probe. Ein Endpunkt kommt also nur zurück, wenn echter Verkehr klappt. Außerdem ist Circuit Breaking mit `ServiceProfile` nicht kompatibel.

Annotation (Präfix `balancer.linkerd.io/`)	Standard
`failure-accrual`	aus, Werte `consecutive` oder `unified`
`failure-accrual-consecutive-max-failures`	`7`
`failure-accrual-consecutive-min-penalty`	`1s`
`failure-accrual-consecutive-max-penalty`	`1m`
`failure-accrual-consecutive-jitter-ratio`	`0.5`

Deine Aufgabe

Setze `max-failures` auf `3`. Was gewinnst du, und was riskierst du bei einem Dienst, der gelegentlich einzelne Fehler liefert? Überlege auch: Was passiert, wenn alle Pods eines Service gleichzeitig ausfallen?

Bei der zweiten Frage hilft die Doku: Sind alle Endpunkte nicht verfügbar, können Requests mit `503` scheitern.

10.2 Lab: Rate Limiting

Rate Limiting schützt den **Server**. Es wird über `HTTPLocalRateLimitPolicy` konfiguriert, die auf einen `Server` zeigt. Jeder `Server` kann nur von einer solchen Policy referenziert werden.

Räume zuerst die Regeln aus Kapitel 6 weg, denn dort existiert ein `Server` mit demselben Namen. Halte auch den Vote-Bot an, damit er die Messung nicht stört.

```
kubectl delete --ignore-not-found -f labs/06-autorisierung/
kubectl -n emojivoto scale --replicas 0 deploy/vote-bot
```

```
# accessPolicy ist Pflichtlektüre: Ohne den Wert würde dieser
Server alles ablehnen,
# denn der Default ist deny.
apiVersion: policy.linkerd.io/v1beta3
kind: Server
metadata:
  name: web-http
  namespace: emojivoto
spec:
  podSelector:
    matchLabels:
      app: web-svc
  port: http
  proxyProtocol: HTTP/1
  accessPolicy: all-unauthenticated
---
apiVersion: policy.linkerd.io/v1alpha1
kind: HTTPLocalRateLimitPolicy
metadata:
  name: web-http
  namespace: emojivoto
```

```
spec:
  targetRef:
    group: policy.linkerd.io
    kind: Server
    name: web-http
  identity:
    requestsPerSecond: 4
  overrides:
    - requestsPerSecond: 6
      clientRefs:
        - kind: ServiceAccount
          namespace: emojivoto
          name: lab-client
```

Achtung

Der `Server` trägt `accessPolicy: all-unauthenticated`. Ohne diese Zeile wäre der Standardwert `deny` aktiv, und du hättest statt eines Limits eine Sperre.

```
kubectl apply -f labs/10-rate-limit/server-und-limit.yaml

kubectl -n emojivoto exec deploy/lab-client -c curl -- sh -c '
  for i in $(seq 10); do
    curl -s -o /dev/null -w "%{http_code} " http://web-svc.
emojivoto/ &
  done
  wait; echo'
```

Prüfpunkt

`lab-client` ist durch sein Override auf 6 Requests pro Sekunde begrenzt. In fünf von fünf Testrunden gingen genau sechs Requests mit `200` durch, die anderen vier endeten mit `429`. Die Reihenfolge ist zufällig, weil die Requests parallel laufen.

Feld	Wirkung
`total.requestsPerSecond`	Gesamtlimit für das Ziel
`identity.requestsPerSecond`	Limit pro Client-Identität

Feld	Wirkung
`overrides`	Eigene Werte für bestimmte ServiceAccounts

Räume danach auf und starte den Vote-Bot wieder:

```
kubectl delete -f labs/10-rate-limit/server-und-limit.yaml
kubectl -n emojivoto scale --replicas 1 deploy/vote-bot
```

10.3 Rate-Limit-bewusstes Load Balancing

Hier steckt eine Falle, die Linkerd 2.20 adressiert. Antworten mit `429` kommen schnell, und das latenzbasierte Load Balancing bevorzugt schnelle Endpunkte. Ein überlasteter Pod kann also *mehr* Verkehr anziehen.

> **Achtung**
>
> Die beiden folgenden Funktionen sind laut Doku **experimentell** und müssen aktiv eingeschaltet werden.

Load Biaser. Die Annotation `balancer.alpha.linkerd.io/penalize-failures: "true"` am Service bewirkt, dass rate-begrenzte Antworten mit einer festen Strafzeit in die Berechnung eingehen, standardmäßig `5s`. Sendet der Server `Retry-After` oder den gRPC-Trailer `grpc-retry-pushback-ms` mit einem höheren Wert, gilt dieser.

Unified Circuit Breaker. `balancer.linkerd.io/failure-accrual: unified` löst zusätzlich aus, wenn die Erfolgsquote unter eine Schwelle fällt, standardmäßig 80 Prozent in einem Fenster von `10s` bei mindestens fünf Antworten. Dabei zählen `429` und `RESOURCE_EXHAUSTED` als Fehler.

10.4 Zusammenfassung

Circuit Breaker schützen den Client vor kaputten Endpunkten. Rate Limits schützen den Server vor zu vielen Clients.

11 Traffic Shifting

Das lernst du hier: Verkehr gewichtet zwischen Versionen verteilen und neue Versionen gezielt testen.

11.1 Das Prinzip

Eine `HTTPRoute` mit einem `Service` als Parent ist eine **Outbound-Route**. Sie sagt den Client-Proxies: Wenn jemand diesen Service anspricht, verteile die Requests wie folgt. Der Service bleibt die stabile Adresse, die Clients kennen, und kein Client muss geändert werden.

```
# Zwei Versionen desselben Dienstes. Der Service bb zeigt nur auf
v1,
# bb-v2 nur auf v2. Die Verteilung übernimmt später eine HTTPRoute.
apiVersion: v1
kind: Namespace
metadata:
  name: traffic-shift-demo
  annotations:
    linkerd.io/inject: enabled
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v1
  namespace: traffic-shift-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: bb
      version: v1
  template:
    metadata:
      labels:
        app: bb
        version: v1
    spec:
      containers:
        - name: terminus
```

```
          image: buoyantio/bb:v0.0.6
          args: ["terminus", "--h1-server-port=8080", "--response-
text=v1"]
          ports:
            - containerPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v2
  namespace: traffic-shift-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: bb
      version: v2
  template:
    metadata:
      labels:
        app: bb
        version: v2
    spec:
      containers:
        - name: terminus
          image: buoyantio/bb:v0.0.6
          args: ["terminus", "--h1-server-port=8080", "--response-
text=v2"]
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: bb
  namespace: traffic-shift-demo
spec:
  selector:
    app: bb
    version: v1
  ports:
    - name: http
      port: 8080
      targetPort: 8080
---
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: bb-v2
  namespace: traffic-shift-demo
spec:
  selector:
    app: bb
    version: v2
  ports:
    - name: http
      port: 8080
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: slow-cooker
  namespace: traffic-shift-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: slow-cooker
  template:
    metadata:
      labels:
        app: slow-cooker
    spec:
      containers:
        - name: slow-cooker
          image: buoyantio/slow_cooker:1.3.0
          command: ["/bin/sh", "-c"]
          args:
            - |
              sleep 5
              /slow_cooker/slow_cooker --qps 10 http://bb:8080
```

```
kubectl apply -f labs/11-traffic/demo.yaml
kubectl -n traffic-shift-demo rollout status deploy/slow-cooker
linkerd viz -n traffic-shift-demo stat --from deploy/slow-cooker
deploy
```

Prüfpunkt

Der gesamte Verkehr landet bei `v1`. `v2` läuft, bekommt aber nichts.

11.2 Lab: zehn Prozent auf die neue Version

```
# 90 % bleiben auf v1, 10 % gehen an v2.
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: bb-route
  namespace: traffic-shift-demo
spec:
  parentRefs:
    - name: bb
      kind: Service
      group: core
      port: 8080
  rules:
    - backendRefs:
        - name: bb
          port: 8080
          weight: 90
        - name: bb-v2
          port: 8080
          weight: 10
```

```
kubectl apply -f labs/11-traffic/route-90-10.yaml
linkerd viz -n traffic-shift-demo stat --from deploy/slow-cooker
deploy
```

Der Service `bb` taucht in `backendRefs` selbst auf. Das bedeutet: 90 Prozent gehen weiter an seine eigenen Endpunkte.

Prüfpunkt

Nach etwa einer Minute teilt sich der Verkehr im Verhältnis 9 zu 1 auf `v1` und `v2` auf. Bei 10 Requests pro Sekunde sind das rund 9 und 1.

Geht alles gut, ziehst du die Gewichte schrittweise nach. Zum Schluss landet alles bei `v2`:

```yaml
# Abschluss des Rollouts: alles an v2.
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: bb-route
  namespace: traffic-shift-demo
spec:
  parentRefs:
    - name: bb
      kind: Service
      group: core
      port: 8080
  rules:
    - backendRefs:
        - name: bb-v2
          port: 8080
          weight: 100
```

```
kubectl apply -f labs/11-traffic/route-100.yaml
```

Der Rückweg ist genauso einfach: Route ändern, fertig. Es muss kein Pod neu starten.

11.3 Lab: Dark Launch per Header

Statt des Zufalls entscheidet hier ein Header. So können Tester die neue Version aufrufen, während alle anderen nichts davon merken.

```yaml
# Dark Launch: Nur Requests mit dem Header x-canary: true sehen v2.
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: bb-route
  namespace: traffic-shift-demo
spec:
  parentRefs:
    - name: bb
      kind: Service
      group: core
      port: 8080
  rules:
    - matches:
        - headers:
            - name: x-canary
```

```
            value: "true"
      backendRefs:
        - name: bb-v2
          port: 8080
    - backendRefs:
        - name: bb
          port: 8080
```

```
kubectl apply -f labs/11-traffic/route-header.yaml
```

Deine Aufgabe

Teste die Route. Du brauchst dafür einen gemeshten Client im Namespace, denn nur ein Outbound-Proxy wendet Routen an. Starte einen eigenen Pod und rufe `http://bb:8080` einmal mit und einmal ohne `-H 'x-canary: true'` auf. Die Antworten verraten mit `v1` oder `v2`, wer geantwortet hat.

11.4 Automatisieren

Gewichte von Hand zu ändern ist fürs Lernen gut. In der Praxis übernimmt das ein Rollout-Werkzeug. Die Linkerd-Doku beschreibt Flagger für automatisierte Canary-Releases anhand von Metriken. Die Grundidee: Das Werkzeug verschiebt das Gewicht nach und nach und rollt zurück, wenn die Kennzahlen schlechter werden.

Hinweis

Ältere Anleitungen nutzen `TrafficSplit` aus der SMI-Erweiterung. Das wird laut Doku noch unterstützt, aber nicht mehr weiterentwickelt. Für Neues nimmst du `HTTPRoute`.

11.5 Aufräumen

```
kubectl delete -f labs/11-traffic/demo.yaml
kubectl delete -f labs/10-circuit-breaker/demo.yaml
kubectl delete -f labs/09-resilienz/demo.yaml
```

11.6 Zusammenfassung

Eine `HTTPRoute` mit `Service`-Parent verteilt Verkehr, ohne Clients zu ändern. Gewichte eignen sich für Canary-Rollouts, Header für gezielte Tests. Alles wirkt nur bei gemeshten Clients.

12 Ingress und Egress

Das lernst du hier: wie Verkehr ins Mesh hinein- und aus ihm herauskommt.

12.1 Ingress

Linkerd bringt keinen Ingress-Controller mit. Es arbeitet mit dem Controller zusammen, den du schon hast. Dafür sind zwei Dinge nötig:

1. den Ingress bei Bedarf für Linkerd konfigurieren,
2. die Ingress-Pods ins Mesh nehmen.

Zwingend ist Letzteres nicht, aber empfohlen: Nur dann gibt es ab dem Eintritt in den Cluster mTLS und Layer-7-Metriken.

Die meisten Controller nimmst du wie jeden anderen Workload mit `linkerd.io/inject: enabled` ins Mesh. Terminiert der Ingress externes TLS, ist diese eingehende Seite für Linkerd undurchsichtiges TCP mit reinen Byte-Metriken. Die ausgehenden Aufrufe an deine Dienste bekommen die volle Behandlung.

Manche Controller brauchen den speziellen Modus `linkerd.io/inject: ingress`. Welche das sind, steht pro Produkt in der Anleitung Handling ingress traffic.

> **Achtung**
>
> Im Ingress-Modus **musst** du den Header `l5d-dst-override` aus eingehendem Client-Verkehr entfernen. Sonst entsteht ein offenes Relay zu internen und externen Zielen. Setze die Annotation außerdem am Workload und nicht am Namespace, damit andere Ressourcen dort normal gemesht werden.

Dieses Buch enthält bewusst kein Ingress-Lab. Die Details hängen vom Produkt ab, und die Doku pflegt sie pro Controller.

12.2 Egress

Egress ist Verkehr aus dem Cluster heraus. Die Ressource dafür heißt `EgressNetwork`. Sie gilt für Clients im selben Namespace, außer sie liegt im global konfigurierten Egress-Namespace.

> **Achtung**
>
> Die Doku warnt ausdrücklich: Kein Service Mesh gibt allein eine harte Sicherheitsgarantie für Egress, denn ein Angreifer kann den Sidecar umgehen. Egress-Kontrolle im Mesh ist Sichtbarkeit und Leitplanke. Wer es strikt braucht, ergänzt Kontrollen auf Netzwerkebene.

12.2.1 Lab: sehen, dann einschränken

Das Lab braucht Internetzugang aus dem Cluster und nutzt `httpbin.org`.

```
# Sichtbar machen: Alles ist erlaubt, wird aber als Egress erfasst.
apiVersion: v1
kind: Namespace
metadata:
  name: egress-test
---
apiVersion: policy.linkerd.io/v1alpha1
kind: EgressNetwork
metadata:
  name: all-egress-traffic
  namespace: egress-test
spec:
  trafficPolicy: Allow
---
apiVersion: v1
kind: Pod
metadata:
  name: client
  namespace: egress-test
  annotations:
    linkerd.io/inject: enabled
    config.linkerd.io/proxy-metrics-hostname-labels: "true"
spec:
  containers:
    - name: client
      image: curlimages/curl:8.23.0
      command: ["sh", "-c", "sleep infinity"]
```

```
kubectl apply -f labs/12-egress/egress-allow.yaml
kubectl -n egress-test wait --for=condition=Ready pod/client --
timeout=120s

# Erstes Terminal: Verkehr erzeugen
kubectl -n egress-test exec -it client -c client -- \
  sh -c 'while sleep 1; do curl -s -o /dev/null -w "%{http_code}\n"
http://httpbin.org/get; done'

# Zweites Terminal: Metriken ansehen
linkerd dg proxy-metrics -n egress-test po/client \
  | grep outbound_http_route_request_statuses_total
```

Prüfpunkt

Die Metrik trägt `parent_kind="EgressNetwork"`, `route_name="http-egress-allow"` und `hostname="httpbin.org"`. Das Hostname-Label ist nur da, weil der Pod die Annotation `config.linkerd.io/proxy-metrics-hostname-labels` trägt. Standardmäßig bleibt es aus, um die Kardinalität niedrig zu halten.

Jetzt drehst du die Logik um:

```
kubectl patch egressnetwork -n egress-test all-egress-traffic \
  -p '{"spec":{"trafficPolicy": "Deny"}}' --type=merge
```

Prüfpunkt

Die Schleife liefert jetzt `403`, und die Route heißt `http-egress-deny`.

Einzelne Ziele gibst du über eine Route frei:

```
# Bei trafficPolicy: Deny kommt nur noch durch, was eine Route
beschreibt.
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: httpbin-get
  namespace: egress-test
spec:
  parentRefs:
```

```
    - name: all-egress-traffic
      kind: EgressNetwork
      group: policy.linkerd.io
      namespace: egress-test
      port: 80
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /get
```

```
kubectl apply -f labs/12-egress/httpbin-get.yaml
```

Prüfpunkt

`/get` antwortet wieder mit `200`. Ein anderer Pfad wie `/headers` sollte gesperrt bleiben.

12.2.2 HTTPS freigeben

HTTPS sieht der Proxy nicht von innen. Er sieht aber den Hostnamen aus dem TLS-Handshake (SNI). Darauf setzt eine `TLSRoute` auf:

```
# HTTPS bleibt für den Proxy undurchsichtig. Freigegeben wird
deshalb per TLS-Hostname (SNI).
# Hinweis: Die Linkerd-Doku zeigt hier noch v1alpha2. Gateway API
1.5.1 liefert im
# Standard-Kanal nur noch v1 aus.
apiVersion: gateway.networking.k8s.io/v1
kind: TLSRoute
metadata:
  name: tls-egress
  namespace: egress-test
spec:
  hostnames:
    - httpbin.org
  parentRefs:
    - name: all-egress-traffic
      kind: EgressNetwork
      group: policy.linkerd.io
      namespace: egress-test
      port: 443
  rules:
```

```
    - backendRefs:
        - kind: EgressNetwork
          group: policy.linkerd.io
          name: all-egress-traffic
```

```
kubectl apply -f labs/12-egress/tls-egress.yaml
kubectl -n egress-test exec client -c client -- \
  curl -s -o /dev/null -w '%{http_code}\n' https://httpbin.org/get
```

Achtung

Die Doku zeigt hier noch `gateway.networking.k8s.io/v1alpha2`. Diese Version liefert Gateway API 1.5.1 im Standard-Kanal nicht mehr aus, deshalb steht im Lab `v1`. Das funktioniert: Vor der Route scheitert `curl` im Test mit Exit-Code 35, danach kommt `200`, und die `TLSRoute` meldet `Accepted`.

Aufräumen:

```
kubectl delete -f labs/12-egress/egress-allow.yaml
```

12.3 Zusammenfassung

Ingress bleibt Sache deines Controllers, und Linkerd schließt daran an. Egress machst du mit `EgressNetwork` sichtbar und schränkst ihn per Route ein. Eine harte Sicherheitsgrenze ist das allein nicht.

13 Multicluster im Überblick

Das lernst du hier: wie Linkerd Cluster verbindet und was du vor dem Start klären musst. Dieses Kapitel ist ein Überblick und kein ausgearbeitetes Lab. Der Abschnitt „Mit kind ausprobieren“ nennt aber, was auf zwei lokalen Clustern funktioniert hat.

13.1 Das Modell

Die Multicluster-Erweiterung spiegelt Services. Ein Service aus dem Cluster `east` erscheint in `west` als lokaler Service. Clients rufen ihn wie gewohnt auf.

Dafür braucht es drei Dinge:

- **gemeinsames Vertrauen:** Alle Cluster teilen denselben Trust Anchor. Das geht nicht mit der Standardinstallation, siehe Kapitel 7.
- **Erreichbarkeit:** Die Cluster müssen sich gegenseitig erreichen können, auch der API-Server des Zielclusters.
- **Zugangsdaten:** Der Spiegelungs-Controller in `west` braucht Leserechte auf Services in `east`.

13.2 Der Ablauf

Die Befehle folgen der aktuellen Edge-Doku. Beachte den Unterschied zu älteren Anleitungen: Der Controller wird über eine Konfigurationsdatei deklariert, und die Verbindung entsteht mit `link-gen`.

```
# Multicluster-Komponenten in beiden Clustern
for ctx in west east; do
  linkerd --context=${ctx} multicluster install \
    | kubectl --context=${ctx} apply -f -
done

# In west einen Controller für den Link "east" deklarieren
cat <<EOF > values.yaml
controllers:
- link:
    ref:
      name: east
EOF
```

```
linkerd --context=west multicluster install -f values.yaml \
  | kubectl --context=west apply -f -

# Link erzeugen: liest east, schreibt nach west
linkerd --context=east multicluster link-gen --cluster-name east \
  | kubectl --context=west apply -f -

# Prüfen
linkerd --context=west multicluster check
linkerd --context=west multicluster gateways

# Einen Service freigeben
kubectl --context=east label svc -n test podinfo mirror.linkerd.io/
exported=true
```

Erreichen sich die Cluster nicht unter derselben Adresse wie von deinem Rechner aus, gibt es `--api-server-address`. Bei lokalen Clustern ist das der Normalfall.

13.3 Mit kind ausprobieren

Zwei kind-Cluster im selben Docker-Netz genügen. Ohne Anpassung scheitert `link-gen` dort aber mit `gateway linkerd-gateway.linkerd-multicluster has no ingress addresses`, denn kind vergibt keine LoadBalancer-Adressen. So ging es im Test:

```
# east: Gateway als NodePort statt LoadBalancer
linkerd --context=east multicluster install \
  --set gateway.serviceType=NodePort \
  | kubectl --context=east apply -f -

EAST_IP=$(docker inspect east-control-plane \
  -f '{{.NetworkSettings.Networks.kind.IPAddress}}')
GW_PORT=$(kubectl --context=east -n linkerd-multicluster \
  get svc linkerd-gateway \
  -o jsonpath='{.spec.ports[?(@.name=="mc-gateway")].nodePort}')

linkerd --context=east multicluster link-gen --cluster-name east \
  --api-server-address https://$EAST_IP:6443 \
  --gateway-addresses $EAST_IP --gateway-port $GW_PORT \
  | kubectl --context=west apply -f -
```

Die Kontexte heißen bei kind `kind-east` und `kind-west`. Im Test wurden sie mit `kubectl config rename-context` umbenannt. Den gemeinsamen Trust

Anchor erzeugst du wie in Kapitel 7 mit `step` und übergibst ihn mit einem eigenen Issuer pro Cluster an `linkerd install`.

Prüfpunkt

In `west` erscheint der Service als `podinfo-east`, und ein gemeshter Client im selben Namespace bekommt von `http://podinfo-east.test:80/` die Antwort `200`. Beide Namespaces waren im Test gemesht.

Achtung

Offen geblieben ist die Liveness-Probe: Im Test zeigte `multicluster gateways` trotz funktionierender Aufrufe `ALIVE False`, und `multicluster check` meldete `liveness checks failed for east`. Die Ursache wurde nicht geklärt. Prüfe das in deiner Umgebung, bevor du dich auf den Status verlässt.

13.4 Federated Services

Die Doku beschreibt außerdem *Federated Services*. Wenn du Services über mehrere Cluster verteilen willst, ist die Anleitung Multi-cluster Federated Services der richtige Einstieg.

13.5 Vor dem Start

Frage	Warum sie wichtig ist
Woher kommt der gemeinsame Trust Anchor?	Nachträglich tauschen ist aufwendig
Dürfen die Cluster sich direkt erreichen?	Bestimmt das Netzwerkmodell
Wer verwaltet die Link-Zugangsdaten?	Sie enthalten eine kubeconfig
Welche Default-Policy gilt?	`cluster-authenticated` erlaubt nur Clients aus demselben Cluster, `all-authenticated` auch andere

13.6 Zusammenfassung

Multicluster spiegelt Services und stützt sich auf einen gemeinsamen Trust Anchor. Plane die Identitäten vor der ersten Installation, das spart später eine Migration.

14 Betrieb

Das lernst du hier: was sich ändert, wenn aus dem Lab echter Betrieb wird.

14.1 Das Release-Modell

Das Projekt veröffentlicht Edge-Releases nach dem Schema `edge-y.m.n`, also Jahr, Monat, laufende Nummer. Sie erscheinen fast wöchentlich und sind **nicht** semantisch versioniert. Die Nummer sagt nichts über Breaking Changes aus. Einige Edge-Releases markieren eine neue logische Version, erkennbar an Tags wie `version-2.20`.

Weg	Pro	Contra
Edge selbst betreiben	Frei verfügbar, aktuell	Du prüfst jedes Release selbst, Breaking Changes möglich
Anbieter-Distribution	Stable-Linie, Support	Kosten und Bedingungen des Anbieters

Das Projekt nennt Edge-Releases grundsätzlich produktionsreif. Stellt sich ein Release als fehlerhaft heraus, wird es als nicht empfohlen gekennzeichnet. Einen konkreten Upgrade-Rhythmus gibt es nicht vor.

Annahme

Eine Faustregel aus Sicht des Autors: Pinne eine Edge-Version, teste sie in einer Vorstufe und aktualisiere in einem festen Rhythmus, zum Beispiel einmal im Monat, statt jedem Release zu folgen. Automatisieren lässt sich das wie bei jeder Abhängigkeit, zum Beispiel mit Renovate.

14.2 Installation mit Helm

Für die Produktion bietet sich Helm an, weil die Werte nachvollziehbar in Git liegen. Die Grundinstallation kennst du aus Kapitel 7. Für HA kommt die Datei `values-ha.yaml` aus dem Chart hinzu:

```
helm fetch --untar linkerd-edge/linkerd-control-plane --version
2026.11.1
```

```
helm install linkerd-control-plane -n linkerd \
  --set-file identityTrustAnchorsPEM=ca.crt \
  --set-file identity.issuer.tls.crtPEM=issuer.crt \
  --set-file identity.issuer.tls.keyPEM=issuer.key \
  -f linkerd-control-plane/values-ha.yaml \
  --version 2026.11.1 \
  linkerd-edge/linkerd-control-plane
```

Mit der CLI lautet das Gegenstück `linkerd install --ha | kubectl apply -f -`.

14.3 Was der HA-Modus tut

Ein Blick in die `values-ha.yaml` von `edge-26.11.1` zeigt:

Einstellung	Wert
`controllerReplicas`	`3`
`enablePodDisruptionBudget`	`true`
`enablePodAntiAffinity`	`true`
`webhookFailurePolicy`	`Fail`
Proxy-Ressourcen	CPU-Request `100m`, Memory-Request `20Mi`, Limit `250Mi`

Achtung

Zwei Folgen musst du kennen. Die Anti-Affinity setzt mindestens **drei Nodes voraus, auf denen die Pods auch laufen dürfen**. Der Lab-Cluster reicht deshalb nicht: Sein Control-Plane-Node hat einen Taint, und im Test blieben beim Upgrade auf HA sechs Pods `Pending`, während die alten weiterliefen. Und `webhookFailurePolicy: Fail` bedeutet: Ist kein Injector gesund, schlagen Deployments fehl.

Hinweis

Der Cluster Autoscaler kann Nodes mit gemeshten Pods standardmäßig nicht herunterskalieren, weil der Proxy einen `tmpfs-emptyDir` nutzt. Abhilfe schafft die Annotation `cluster-autoscaler.kubernetes.io/safe-to-evict: "true"` am Workload oder `--skip-nodes-with-local-storage=false` am Autoscaler.

14.4 Upgrades

Die Reihenfolge ist immer dieselbe: CLI, Control Plane, Erweiterungen, Data Plane.

```
# Vorher: Zustand prüfen, Release Notes lesen
linkerd check
linkerd check --proxy

# CLI (Zielversion setzen)
export LINKERD2_VERSION=edge-26.11.1
curl --proto '=https' --tlsv1.2 -sSfL https://run.linkerd.io/
install-edge | sh

# Control Plane
linkerd upgrade --crds | kubectl apply -f -
linkerd upgrade | kubectl apply -f -
linkerd prune | kubectl delete -f -
linkerd check

# Erweiterungen
linkerd viz install | kubectl apply -f -
linkerd viz prune | kubectl delete -f -

# Data Plane
kubectl -n <namespace> rollout restart deploy
linkerd check --proxy
```

Mit Helm läuft der Control-Plane-Teil so:

```
helm repo update
helm upgrade linkerd-crds linkerd-edge/linkerd-crds -n linkerd --
version 2026.11.1
helm upgrade linkerd-control-plane linkerd-edge/linkerd-control-
plane \
```

```
  -n linkerd --version 2026.11.1 \
  --reset-values -f values.yaml --rollback-on-failure
```

`--rollback-on-failure` ist der Name in Helm 4. Helm 3 nennt das Flag `--atomic`. Helm 4.3 nimmt das alte Flag noch an, führt es in der Hilfe aber nicht mehr auf.

Version Skew. Die Control Plane darf der Data Plane um bis zu eine Hauptversion voraus sein. Mehr wird nicht unterstützt. Du kannst die Data Plane also gestaffelt aktualisieren, darfst das aber nicht endlos aufschieben.

Achtung

Rechne bei Major-Upgrades mit Pod-Neustarts. Laut den Release Notes von Buoyant werden beim Wechsel auf 2.20 injizierte Workloads neu ausgerollt, weil sich die Pod-Form ändert. Prüfe vor jedem Upgrade außerdem die unterstützten Kubernetes-Versionen. Die Tabelle nennt für 2.20 ein Minimum von 1.31, für 2.19 noch 1.22.

14.5 Checkliste für die Produktion

Diese Liste ist eine Empfehlung des Autors.

- Eigene Zertifikate, Issuer mit automatischer Rotation (Kapitel 7)
- Warnung vor dem Ablauf von Issuer und Trust Anchor
- HA-Modus mit mindestens drei Nodes
- Ressourcen für Proxies bewusst gesetzt
- Eigenes Prometheus statt des Viz-Prometheus
- `Server` und `AuthorizationPolicy` über Audit eingeführt (Kapitel 6)
- Default-Policy bewusst gewählt
- Alte `ServiceProfile` geprüft und bewusst entschieden, ob sie bleiben
- Version gepinnt, Upgrade-Prozess dokumentiert und geübt
- Gateway-API-Version kompatibel zur Linkerd-Version

14.6 Zusammenfassung

Betrieb heißt vor allem: Versionen bewusst führen, Zertifikate überwachen, HA-Vorgaben einhalten und Upgrades in fester Reihenfolge fahren.

15 Fehlersuche

Das lernst du hier: ein festes Vorgehen für Störungen im Mesh und die häufigsten Fehlerbilder.

15.1 Das Vorgehen

Springe bei einer Störung nicht sofort in die Logs. Arbeite von außen nach innen.

1. **Zustand des Meshes:** `linkerd check`, dann `linkerd check --proxy -n <ns>`
2. **Wer leidet:** `linkerd viz -n <ns> stat deploy`
3. **Welche Richtung:** `stat-inbound` und `stat-outbound` vergleichen
4. **Welche Regel:** `linkerd viz authz -n <ns> deploy/<name>`
5. **Was genau:** `linkerd viz -n <ns> tap deploy/<name>`
6. **Was denkt der Proxy:** `linkerd diagnostics policy` und `proxy-metrics`
7. **Erst jetzt:** `kubectl logs <pod> -c linkerd-proxy`

15.2 Häufige Fehlerbilder

Die Tabelle fasst Hinweise aus der Doku und den vorherigen Kapiteln zusammen.

Symptom	Wahrscheinliche Ursache	Prüfen und beheben
Pod nicht im Mesh	Annotation fehlt, oder Pod nicht neu gestartet	`kubectl get pod -o yaml`, Rollout neu starten
Plötzlich alles `403`	`Server` ohne Autorisierung, `accessPolicy` ist `deny`	`viz authz`, Audit-Modus nutzen
Pods werden nicht Ready	`Server` auf Probe-Port, Probes nicht erlaubt	Route und `NetworkAuthentication` für Probes
Pfad nicht erreichbar	Route am `Server` deckt Pfad nicht ab	`inbound_http_route_not_found_total`
Prometheus leer	Scrapes durch Policy gesperrt	`linkerd viz check --proxy`, `linkerd viz allow-scrapes`

Symptom	Wahrscheinliche Ursache	Prüfen und beheben
Annotationen wirken nicht	Client ungemesht, Pod nicht neu gestartet oder Annotation an der falschen Route	`linkerd diagnostics policy`, `kubectl get pod -o yaml`
10 Sekunden Verzögerung	Protokollerkennung läuft ins Timeout	Protokoll deklarieren, siehe unten
Keine HTTP-Metriken	Anwendung spricht selbst TLS	Erwartetes Verhalten, nur TCP-Metriken
Installation scheitert	Kubernetes-Version nicht unterstützt	`linkerd check --pre`, Versionstabelle
Neue Pods hängen	Injector nicht gesund, `webhookFailurePolicy: Fail`	`kubectl -n linkerd get pods`
Control Plane unvollständig	HA mit weniger als drei Nodes	Nodes ergänzen oder Replikas reduzieren

15.3 Der 10-Sekunden-Fall

Sendet der Client nach dem Verbindungsaufbau nichts, wartet der Proxy bis zu 10 Sekunden auf Bytes und behandelt die Verbindung erst dann als TCP. In den Proxy-Logs steht dann `protocol detection timed out after 10s`. Typisch ist das bei Protokollen, in denen der Server zuerst spricht.

Besser ist es, dem Proxy das Protokoll zu sagen. Dafür gibt es das Feld `appProtocol` am Service-Port:

`appProtocol`	Behandlung
`linkerd.io/opaque`	Undurchsichtiges TCP
`linkerd.io/tcp`	Alias für `linkerd.io/opaque`
`http`	HTTP/1
`kubernetes.io/h2c`	HTTP/2

Andere gesetzte Werte behandelt Linkerd als undurchsichtiges TCP.

Ein Teil der Ports ist schon als opaque eingetragen:

Protokoll	Ports	In der Standardliste?
SMTP	25, 587	Ja
MySQL	3306	Ja

Protokoll	Ports	In der Standardliste?
MySQL mit Galera	3306, 4444, 4567, 4568	Teilweise, nicht 4567 und 4568
PostgreSQL	5432	Ja
Redis	6379	Ja
ElasticSearch	9300	Ja
Memcache	11211	Ja
NATS	4222, 6222, 8222	Nein

In drei Fällen hilft `appProtocol` nicht, und du nimmst die Annotation `config.linkerd.io/opaque-ports`: wenn ein Pod Verkehr von ungemeshten Clients bekommt, bei Headless Services oder direkten Pod-Aufrufen und bei `EgressNetwork`-Ressourcen.

Achtung

Die Annotation ersetzt die Standardliste, statt sie zu ergänzen. Wer nur `4567` einträgt, nimmt die übrigen Ports aus der Liste.

15.4 Mehr Details aus dem Proxy

Für eine Analyse hilft ein höherer Log-Level am Workload. Die Annotation wird beim Injizieren gelesen, wirkt also erst nach einem Neustart des Pods:

```
metadata:
  annotations:
    config.linkerd.io/proxy-log-level: warn,linkerd=debug
```

Für die Netzwerkanalyse gibt es den Debug-Sidecar aus Kapitel 5. Nimm beides nach der Analyse wieder heraus.

15.5 Eine Übung zum Abschluss

Deine Aufgabe

Lass dir von jemandem einen dieser Fehler in dein Lab einbauen, ohne zu verraten, welchen:

1. Ein `Server` auf einem Port ohne Autorisierung.
2. Eine `HTTPRoute` am `Server`, die einen genutzten Pfad nicht abdeckt.
3. Ein Deployment mit neuer Annotation, das nicht neu gestartet wurde.

Finde die Ursache nur mit dem Vorgehen von oben. Notiere, bei welchem Schritt du sie erkannt hast.

15.6 Zusammenfassung

Arbeite von außen nach innen: erst `check`, dann Metriken, dann einzelne Requests, zuletzt Logs. Häufige Ursachen sind ein `Server` ohne Regel, ein Pod ohne Neustart oder ein Protokoll, das der Proxy nicht erkennt.

Damit bist du den Weg vom ersten Sidecar bis zum Betrieb einmal gegangen. Der nächste sinnvolle Schritt ist ein echter Test: Nimm einen unkritischen Dienst aus deinem Alltag, führe ihn im Audit-Modus ein und beobachte eine Woche lang, was du lernst.

Anhang A: Spickzettel

Installation

```
linkerd check --pre
linkerd install --crds | kubectl apply -f -
linkerd install | kubectl apply -f -
linkerd viz install | kubectl apply -f -
linkerd check
```

Pods ins Mesh nehmen

```
kubectl get -n <ns> deploy -o yaml | linkerd inject - | kubectl
apply -f -
kubectl annotate ns <ns> linkerd.io/inject=enabled
kubectl -n <ns> rollout restart deploy
```

Diagnose

```
linkerd check --proxy -n <ns>
linkerd viz -n <ns> stat deploy
linkerd viz -n <ns> stat-inbound deploy/<name>
linkerd viz -n <ns> stat-outbound deploy/<name>
linkerd viz -n <ns> top deploy/<name>
linkerd viz -n <ns> tap deploy/<name>
linkerd viz -n <ns> edges deployment
linkerd viz authz -n <ns> deploy/<name>
linkerd diagnostics proxy-metrics -n <ns> deploy/<name>
linkerd diagnostics policy -n <ns> svc/<name> <port> -o json
```

Annotationen

Annotation	Sitzt an	Zweck
`linkerd.io/inject`	Namespace, Pod	`enabled` oder `ingress`
`config.linkerd.io/default-inbound-policy`	Namespace, Pod	Default-Policy

Annotation	Sitzt an	Zweck
`config.linkerd.io/opaque-ports`	Pod, `EgressNetwork`	Ports ohne Protokollerkennung
`config.linkerd.io/proxy-log-level`	Pod	Log-Level des Proxys
`config.linkerd.io/proxy-metrics-hostname-labels`	Namespace, Pod	Hostname in Metriken
`retry.linkerd.io/http`, `grpc`, `limit`, `timeout`	Service, Route	Retries
`timeout.linkerd.io/request`, `response`, `idle`	Service, Route	Timeouts
`balancer.linkerd.io/failure-accrual`	Service	Circuit Breaker

Anhang B: Prüfstand dieser Auflage

Dieser Anhang hält fest, was für diese Auflage am 6. November 2026 tatsächlich gelaufen ist und was nicht.

Testumgebung

Komponente	Version
VM	Multipass, Ubuntu 26.04, arm64, 4 vCPUs, 8 GB RAM, 20 GB Platte
Linkerd	CLI und Control Plane `edge-26.11.1`
kind / Node-Image	0.33.0 / Kubernetes 1.35.8
kubectl / Helm	1.35.10 / 4.3.1
Docker	29.1.3
step	0.31.0

Control Plane und Viz belegten zusammen mit dem Cluster rund 2 GB Speicher.

Im Cluster gelaufen

Kapitel	Ergebnis
2 Installation	`check --pre`, `check` und `viz check` fehlerfrei, auch mit Gateway API 1.5.1
4 Emojivoto	Inject, `stat`, `tap`, Fehlerbild wie beschrieben
5 mTLS	`edges`, `tap`, `tshark`, ungemeshter Aufruf mit `200` und `no_tls_from_remote`
6 Autorisierung	Audit, Regeln, Durchsetzung, `grpc-status: 7`, Routen mit `200`, `403` und `404`
7 Zertifikate	`step`, Helm-Installation, Fingerprint-Vergleich identisch
8 Beobachten	`stat-inbound`, `stat-outbound`, Metriken, PromQL-Abfrage, `allow-scrapes` mit Deny-Policy
9 Resilienz	69 Prozent, mit Retries 97,3 Prozent, Timeout `504`

Kapitel	Ergebnis
10 Circuit Breaker und Rate Limit	Breaker wirkt, 6 von 10 in fünf von fünf Runden
11 Traffic Shifting	9 zu 1, 100 Prozent, Header-Routing
12 Egress	HTTP erlaubt und gesperrt, Route, `TLSRoute v1`
13 Multicluster	Spiegelung und Aufruf `200` über das Gateway, Probe siehe unten
14 Betrieb	Upgrade-Befehle, `prune`, Helm-Upgrade, HA-Fehlerbild

In der ersten Fassung falsch

Diese Fehler hat erst der Lauf im Cluster aufgedeckt. Die Schemaprüfung konnte sie nicht finden.

Stelle	Fehler	Korrektur
Kapitel 2	Nodes direkt nach `kind create` nicht `Ready`	`kubectl wait` ergänzt
Kapitel 2	Prüfsummendatei gibt es nicht	Digest über die GitHub-API
Kapitel 5	TLS auf Port 8080 behauptet, dort läuft Klartext	Port 4143, Klartext-Gegenprobe
Kapitel 5	`grep voting` traf während des Rollouts zwei Pods	`deploy/voting`
Kapitel 5	`no_tls_from_remote` durch Probes, Emojivoto hat keine	Text korrigiert
Kapitel 7	Nach `kind delete` kein Kontext gesetzt	`use-context` ergänzt
Kapitel 8	Metrikname falsch, Alarm wäre stumm geblieben	Präfix `control_`
Kapitel 8	`viz check` ohne `--proxy` warnt nicht	`--proxy -n <ns>`
Kapitel 9	Eine Minute reicht nicht für 97 Prozent	Messreihe angegeben

Stelle	Fehler	Korrektur
Kapitel 9, 15	`ServiceProfile` hebelt Routen aus, nicht reproduzierbar	Text und Übung geändert
Kapitel 10	50 Prozent Fehler ohne Breaker, gemessen 1 und 67 Prozent	Prüfpunkt neu
Kapitel 14	Helm-Upgrade ohne `-n linkerd` scheitert mit `has no deployed releases`, `--atomic` in Helm 4 alt	Befehle korrigiert
Kapitel 14	Lab-Cluster reicht nicht für HA	Hinweis ergänzt

Nicht gelaufen oder offen

Achtung

Diese Punkte sind nicht belegt. Geh hier von Doku-Stand aus.

Stelle	Stand
Kapitel 2, `curl | sh`	Installationsskript nicht ausgeführt, CLI als Binärdatei installiert
Kapitel 2, 4	macOS, Colima, Browser, `viz dashboard`, Port-Forward nicht getestet
Kapitel 4, 8	`viz top` startet nur im Terminal, Ausgabe nicht ausgewertet
Kapitel 6	Aufgabe zu `lab-client` und `/api/vote` nicht durchgespielt
Kapitel 7	cert-manager, trust-manager und Trust-Anchor-Rotation nur beschrieben
Kapitel 8	Alarm nur als Abfrage gelaufen, nie im Fehlerfall
Kapitel 9	`retry.linkerd.io/timeout`, GET-Aufgabe, Annotationen direkt am `Service`
Kapitel 10	Load Biaser und Unified Circuit Breaker nicht getestet

Stelle	Stand
Kapitel 12	Kein Ingress-Lab
Kapitel 13	Liveness-Probe `ALIVE False` ungeklärt, Federated Services nicht getestet
Kapitel 14	Upgrade nur auf dieselbe Version, kein echter Versionssprung. `linkerd install --ha` nicht ausgeführt, HA nur per Helm und dort nicht vollständig gestartet
Kapitel 15	Protokollerkennung und `opaque-ports` nur aus der Doku
Allgemein	Alles lief einmal in dieser einen Umgebung, nicht auf Linux amd64 oder macOS

Abweichungen von der Linkerd-Doku

An diesen Stellen weicht das Buch bewusst ab, und die Läufe bestätigen die Abweichung:

Thema	Doku zeigt	Buch nutzt	Beleg
`Server`-Version	`v1beta1`	`v1beta3`	In der CRD deprecated, Lab lief
Autorisierung	`ServerAuthorization`	`AuthorizationPolicy`	Lab lief
`HTTPRoute`	teils Linkerd-eigene Gruppe	Gateway API, `v1`	Lab lief
`TLSRoute`	`v1alpha2`	`v1`	Lab lief, Status `Accepted`
kind-Node	keine Angabe	`v1.35.8`	Lab lief, kind-Default 1.37 liegt über dem Maximum

Selbst nachprüfen

`pruefung/LIESMICH.txt` beschreibt, wie du die Referenzdaten für die Schemaprüfung holst. Danach genügt:

```
ruby pruefung/pruefe-labs.rb <refdata-Verzeichnis>
```

Die Laufprotokolle der Tests liegen unter `pruefung/laufprotokolle/`. Die Schemaprüfung prüft nur die Form der Manifeste. Die Fehler aus der Tabelle oben zeigen, warum das einen Lauf im Cluster nicht ersetzt.

Anhang C: Quellen

Stand aller Quellen: 6. November 2026.

Linkerd

- Releases: https://linkerd.io/releases/
- Release `edge-26.11.1`: https://github.com/linkerd/linkerd2/releases/tag/edge-26.11.1
- Ankündigung 2.20: https://linkerd.io/2026/06/23/announcing-linkerd-2.20/
- Getting Started: https://linkerd.io/2-edge/getting-started/
- Architektur: https://linkerd.io/2-edge/reference/architecture/
- Kubernetes-Versionen: https://linkerd.io/2-edge/reference/k8s-versions/
- Gateway API: https://linkerd.io/2-edge/features/gateway-api/
- Automatisches mTLS: https://linkerd.io/2-edge/features/automatic-mtls/
- Authorization Policy: https://linkerd.io/2-edge/reference/authorization-policy/
- Retries: https://linkerd.io/2-edge/reference/retries/
- Timeouts: https://linkerd.io/2-edge/reference/timeouts/
- Circuit Breaking: https://linkerd.io/2-edge/reference/circuit-breaking/
- Rate Limiting: https://linkerd.io/2-edge/reference/rate-limiting/
- Rate Limits im Load Balancing: https://linkerd.io/2-edge/tasks/rate-limit-aware-load-balancing/
- Traffic Shifting: https://linkerd.io/2-edge/tasks/traffic-shifting/
- Ingress: https://linkerd.io/2-edge/tasks/using-ingress/
- Egress: https://linkerd.io/2-edge/tasks/managing-egress-traffic/
- Multicluster: https://linkerd.io/2-edge/tasks/multicluster/
- Helm: https://linkerd.io/2-edge/tasks/install-helm/
- HA: https://linkerd.io/2-edge/features/ha/
- Upgrades: https://linkerd.io/2-edge/tasks/upgrade/
- Protokollerkennung: https://linkerd.io/2-edge/features/protocol-detection/
- Troubleshooting: https://linkerd.io/2-edge/tasks/troubleshooting/
- Quellcode: https://github.com/linkerd/linkerd2/tree/edge-26.11.1

Weitere

- Gateway API Releases: https://github.com/kubernetes-sigs/gateway-api/releases
- kind Releases: https://github.com/kubernetes-sigs/kind/releases
- bb: https://github.com/BuoyantIO/bb
- Buoyant Enterprise for Linkerd 2.20: https://docs.buoyant.io/release-notes/buoyant-enterprise-linkerd/enterprise-2.20.0/